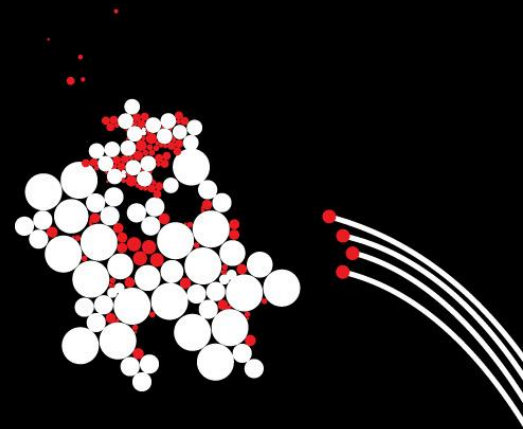


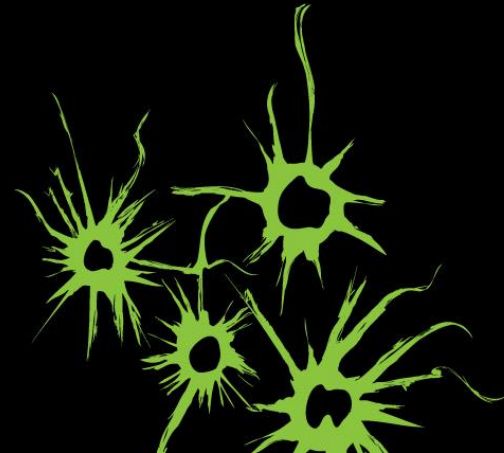
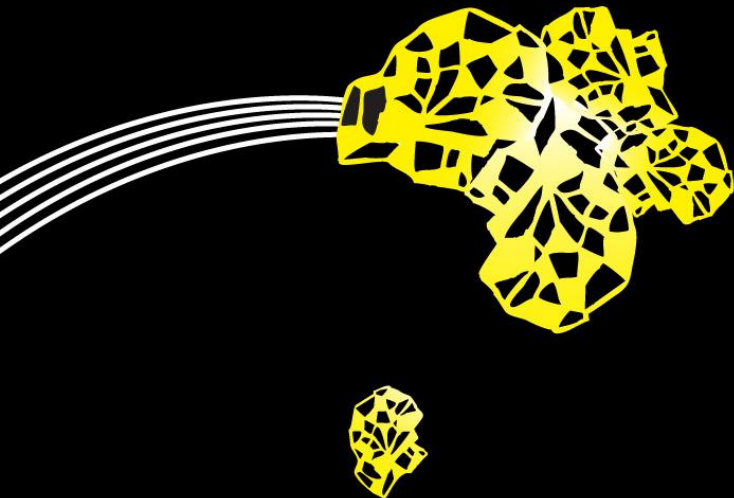
UNIVERSITEIT TWENTE.

DS & AI [2202200020] SEEING THROUGH THE HYPE



DATA QUALITY – LECTURE 2 & 3 DETECTION, CLEANING & MITIGATION OF DATA QUALITY PROBLEMS

MAURICE VAN KEULEN



Grade for module part “DQ”
is 70% of grade of
DQ&Ethics study unit

OVERVIEW OF DQ PART

3 Lectures

- Lec 1: What is DQ and Neuro-Symbolic AI?
 - Lab 1: Sensitivity of ML to DQ problems
- Lec 2&3: Detection, cleaning & mitigation of DQ problems
 - Lab 2: Detection & cleaning (mostly on-paper)

DQ assessment

- Test: 80%; Lab assignments 20%
- Lab 1: 30 points; Lab 2: 70 points

STUDY MATERIAL OF THIS LECTURE

Book (available as e-book through UT's library)

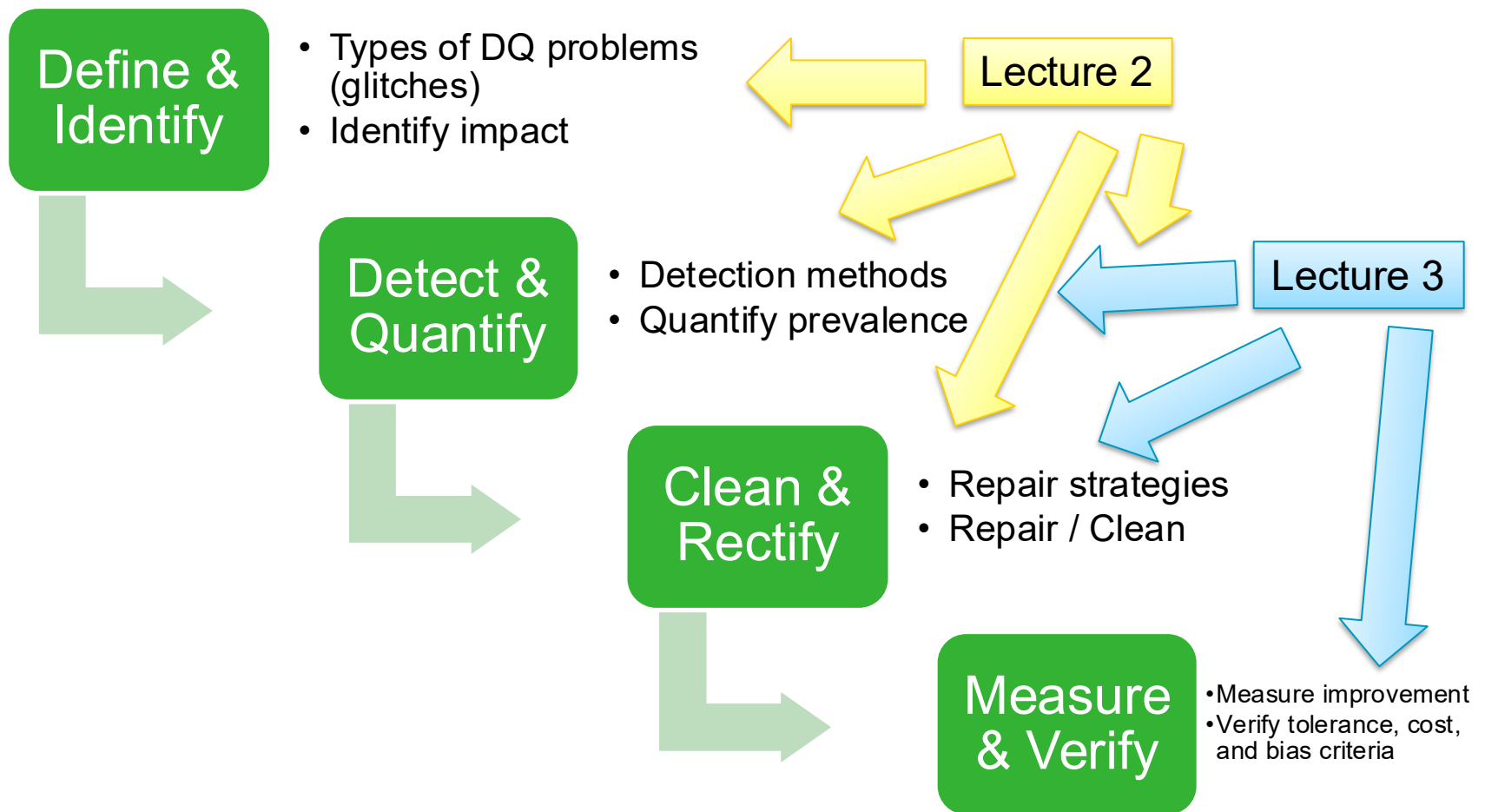
- Shazia Sadiq ed. (2013). *Handbook of data quality: research and practice*. Heidelberg, Germany: Springer. ISBN 978-3-642-36256-9. DOI [10.1007/978-3-642-36257-6](https://doi.org/10.1007/978-3-642-36257-6)
 - Chp II-1: "Data Warehouse Quality: Summary and Outlook" (but part of this chapter is also used in Lecture 1)
 - Chp II-3: "Data Glitches: Monsters in Your Data"
 - Chp III-1: "Generic and Declarative Approaches to Data Quality Management" (Except sections 4.2 and 5; other approach for 3.1)
 - Chp III-3: "A Practical Guide to Entity Resolution with OYSTER"
- Emmanuel, T., et al. A survey on missing data in machine learning. J Big Data 8, 140 (2021). DOI 10.1186/s40537-021-00516-9

And the slides of course

- Part of these chapters, I will address in the lecture
... but I expect you to have studied these chapters in full for the exam

DATA QUALITY PROCESS

SOURCE: FIGURE 1 PAGE 165



COMPLEX DATA GLITCHES

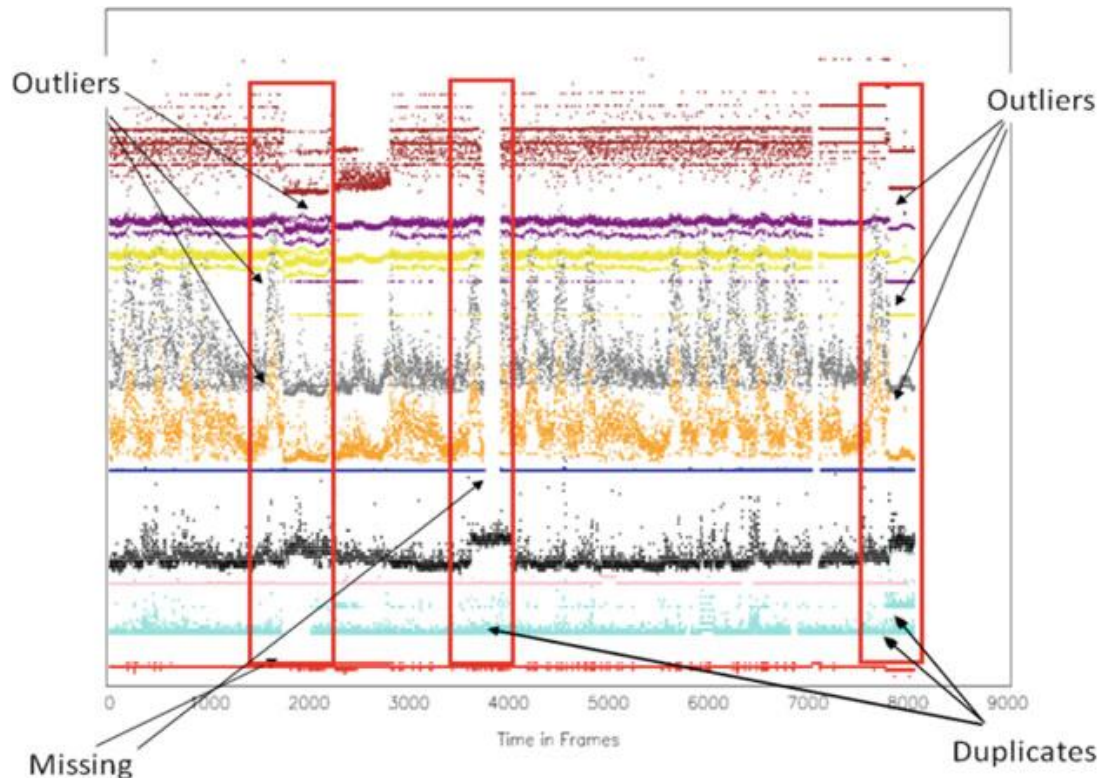
- Missing data (values, records, attributes)
 - May easily go unnoticed; can create bias
- Inconsistent erroneous data (values, references)
 - Constraints; exploit redundancy
- Anomalies and outliers (values, records)
 - Unexpected, doubtful
- (Semantic) duplicates (records)
 - One entity in the real world; two or more in data
- Undocumented data
 - What does “123-44-1212|3.59|2|145” mean?
 - Subtle details matter (e.g. owner of parcel of land)

COMPLICATING FACTORS FOR DETECTION

- Relevance
 - Meaning, severity, treatment varies over domains
- Ambiguity
 - Boundary between good and bad unclear
- Complex dependencies
 - One type of glitch can mask another (e.g. missing values can mask a duplicate)
 - If duplicate, then inconsistent
- Dynamic
 - Not only new glitches appear, also type of glitches changes

GLITCHES OFTEN OCCUR IN PATTERNS

SOURCE: FIG 2 PAGE 169



10 attributes related to performance of a network element in an IP network over a period of time

CLASSIFICATION OF GLITCH COMPLEXES

- Multi-type glitch
 - Value has multiple glitch types associated with it
- Concomitant glitches
 - Two or more values have glitches and appear in same record
- Multi-occurrent glitch
 - Glitch type occurring with two or more values in data

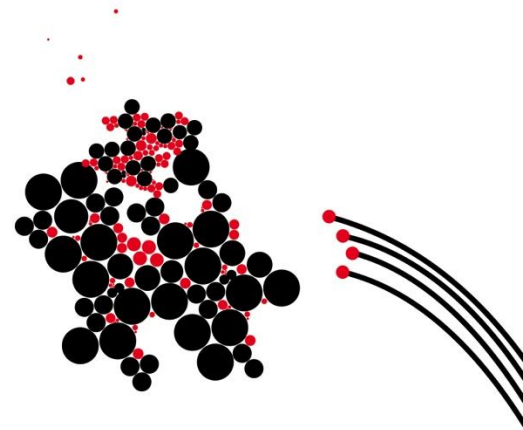
QUANTIFYING THE NUMBER OF GLITCHES

Straightforward method of quantification

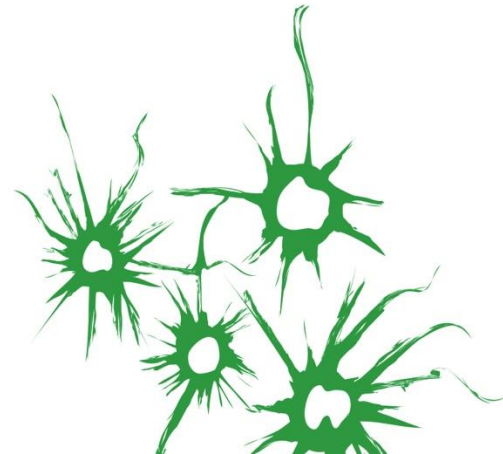
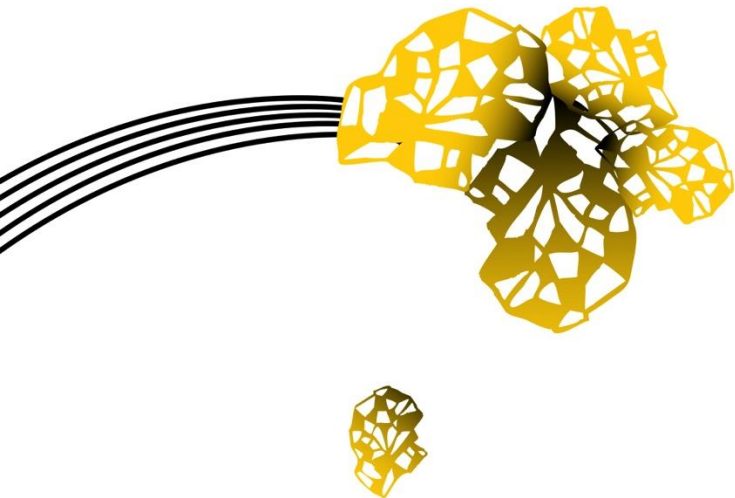
- Per value
 - Glitch signature: vector with per glitch type indication of its presence (0/1 or probabilities)
 - Determine weights per glitch type
 - Glitch score of a value: weighted sum
- Global
 - Sum of all glitch scores

COMPLEX DATA GLITCHES

- **Missing data** (values, records, attributes)
 - May easily go unnoticed; can create bias
- Inconsistent erroneous data (values, references)
 - **Constraints**; exploit redundancy
- **Anomalies** and outliers (values, records)
 - Unexpected, doubtful
- (Semantic) **duplicates** (records)
 - One entity in the real world; two or more in data
- Undocumented data
 - What does “123-44-1212|3.59|2|145” mean?
 - Subtle details matter (e.g. owner of parcel of land)



CONSTRAINTS

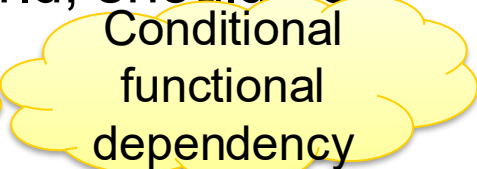


WHAT IS A CONSTRAINT

- Constraint: any rule that should hold in your data
 - Data is a simplified abstracted representation of the real world, so any rule that holds in the real world, should hold for your data
 - Age of a human < 125
 - Social security numbers are unique for persons
 - In NL, postal code + house number determines street and city
- Also called integrity constraint or semantic constraint
- Soft constraint: rule that almost always holds, but there can be exceptions
 - Name of employee is unique within the organisation
 - Person enrolled in course implies person is a student



Key
constraint



Conditional
functional
dependency

FUNCTIONAL DEPENDENCIES

- Notation: $X \rightarrow Y$
(attributes X determine one value for attributes Y)
- Example: $Snum \rightarrow Name$

Ambiguous: is there a glitch in the Snum or are these records duplicates?

Students	
Snum	Name
101	John Bell
101	Jonathan Bell
104	Claire Stevens
107	Pat Norton

- Formal notation in predicate logic
 - $\forall x \forall y \forall z (Students(x,y) \wedge Students(x,z) \Rightarrow y=z)$

REFERENTIAL INTEGRITY CONSTRAINTS

- Notation: $\text{Enrollment}[\text{Snum}] \subseteq \text{Students}[\text{Snum}]$

Students		Enrollment	
Snum	Name	Snum	Course
101	John Bell	101	DSAI
101	Jonathan Bell	101	DSAI
104	Claire Stevens	104	D&I
107	Pat Norton	101	D&I

- Formal notation in predicate logic
 - $\forall x \forall y (\text{Enrollment}(x,y) \Rightarrow \exists z \text{Students}(x,z))$

CONSISTENCY

Database D is consistent if and only if it satisfies all constraints Σ , i.e., $D \models \Sigma$

Better to quantify

- Count the number of violations of each constraint
- Then sum over all constraints (possible weighted)

CONDITIONAL DEPENDENCIES

Staff

Num	Name	Type	Street	City	Country	Zip
1	john	admin	First	Miami	US	33114
2	nill	journalist	Bronson	Miami	US	33114
3	nick	journalist	Glebe	Tampa	US	33605
4	ana	admin	Crow	Glasgow	UK	G11 7HS
5	mary	journalist	Main	Glasgow	UK	G11 7HS

Journalist

Num	Role
1	news
2	news
2	columnist
3	columnist

- $\text{Num} \rightarrow \text{Name, Type, Street, City, Zip}$
 $\text{Zip, Country} \rightarrow \text{City}$
 $\text{Journalist}[\text{Num}] \subseteq \text{Staff}[\text{Num}]$
- But not
 - $\text{Zip} \rightarrow \text{Street}$
(only in UK, not in US)
 - Every journalist in Staff has role in Journalist

CONDITIONAL FUNCTIONAL DEPENDENCIES (CFD)

Conditional Functional Dependency (CFD)

- Pair $(X \rightarrow Y, T_p)$
 T_p is a pattern tableau

- $(\text{Country}, \text{Zip} \rightarrow \text{Street}, \text{City}, T_1)$

T_1

Country	Zip	Street	City
UK	-	-	-

Staff

Num	Name	Type	Street	City	Country	Zip
1	john	admin	First	Miami	US	33114
2	nill	journalist	Bronson	Miami	US	33114
3	nick	journalist	Glebe	Tampa	US	33605
4	ana	admin	Crow	Glasgow	UK	G11 7HS
5	mary	journalist	Main	Glasgow	UK	G11 7HS

Journalist

Num	Role
1	news
2	news
2	columnist
3	columnist

CONDITIONAL INCLUSION DEPENDENCIES (CIND)

T₂

Num	Type	Num
-	journalist	-

Conditional Functional Dependency (CIND)

T₃

- Pair $(R_1[X;X_p] \subseteq R_2[Y;Y_p], T_p)$

T_p is a pattern tableau

Num	Num	Type
-	-	journalist

- $(\text{Staff}[\text{Num};\text{Type}] \subseteq \text{Journalist}[\text{Num};\text{nil}], T_2)$
- $(\text{Journalist}[\text{Num};\text{nil}] \subseteq \text{Staff}[\text{Num};\text{Type}], T_3)$

Staff

Num	Name	Type	Street	City	Country	Zip
1	john	admin	First	Miami	US	33114
2	nill	journalist	Bronson	Miami	US	33114
3	nick	journalist	Glebe	Tampa	US	33605
4	ana	admin	Crow	Glasgow	UK	G11 7HS
5	mary	journalist	Main	Glasgow	UK	G11 7HS

Journalist

Num	Role
1	news
2	news
2	columnist
3	columnist

CONDITIONAL DEPENDENCIES

Num	Num	Type
-	-	journalist

It may be ambiguous where the fault lies when a dependency is violated

- Example: $(\text{Journalist}[\text{Num}; \text{nil}] \subseteq \text{Staff}[\text{Num}; \text{Type}], T_3)$
- Is $\text{Staff}[1; \text{admin}]$ wrong or $\text{Journalist}[1; \text{news}]$?
- Hampers counting: how many violations in Staff?



Num	Name	Type	Street	City	Country	Zip
1	john	admin	First	Miami	US	33114
2	nill	journalist	Bronson	Miami	US	33114
3	nick	journalist	Glebe	Tampa	US	33605
4	ana	admin	Crow	Glasgow	UK	G11 7HS
5	mary	journalist	Main	Glasgow	UK	G11 7HS



Num	Role
1	news
2	news
2	columnist
3	columnist

DISCOVERING CFDS AND CINDS

There are algorithms that can automatically discover CFDs and CINDs from a set of tables!

- There are also algorithms that can discover them even if they do not hold perfectly in your data, but are valid upto a certain threshold of violations
- Assuming we trust these discovered rules as true constraints, they can then be used as soft constraints to flag ‘possibly’ violating records and values

SOLUTION DIRECTION

REPAIRS AND CONSISTENT ANSWERS

REMEMBER THIS EXAMPLE OF A FUNCTIONAL DEPENDENCY

- Notation: $X \rightarrow Y$
(attributes X determine one value for attributes Y)
- Example: $Snum \rightarrow Name$

Ambiguous: is there a glitch in the Snum or are these records duplicates?

Students	
Snum	Name
101	John Bell
101	Jonathan Bell
104	Claire Stevens
107	Pat Norton

- Formal notation in predicate logic
 - $\forall x \forall y \forall z (Students(x,y) \wedge Students(x,z) \Rightarrow y=z)$

SOLUTION DIRECTION

REPAIRS AND CONSISTENT ANSWERS

A *minimal repair*, or simply, repair, is a minimal change to the database to remove the constraint violation.

Four possible minimal repairs

Students

Snum	Name
101	John Bell
101	Jonathan Bell
104	Claire Stevens
107	Pat Norton

Snum	Name
101	John Bell
101	Jonathan Bell
104	Claire Stevens
107	Pat Norton

Snum	Name
101	John Bell
102	
101	Jonathan Bell
104	Claire Stevens
107	Pat Norton

Snum	Name
101	John Bell
101	Jonathan Bell
102	
104	Claire Stevens
107	Pat Norton

SOLUTION DIRECTION

REPAIRS AND CONSISTENT ANSWERS

Consistent Query Answering (CQA) is a method to obtain all query answers that are invariant to any possible way of restoring consistency by performing minimal repairs.

Snum	Name
101	John Bell
102	
101	Jonathan Bell
104	Claire Stevens
107	Pat Norton

Snum	Name
101	John Bell
101	Jonathan Bell
102	
104	Claire Stevens
107	Pat Norton

Students	
Snum	Name
101	John Bell
101	Jonathan Bell
104	Claire Stevens
107	Pat Norton

Snum	Name
101	John Bell
101	Jonathan Bell
104	Claire Stevens
107	Pat Norton

SELECT *
FROM Students

CQA

Snum	Name
104	Claire Stevens
107	Pat Norton

SOLUTION DIRECTION

REPAIRS AND CONSISTENT ANSWERS

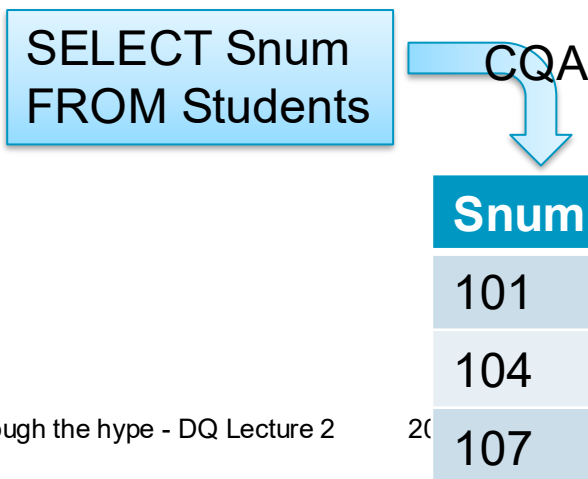
Consistent Query Answering (CQA) is a method to obtain all query answers that are invariant to any possible way of restoring consistency by performing minimal repairs.

Snum	Name
101	John Bell
102	
101	Jonathan Bell
104	Claire Stevens
107	Pat Norton

Snum	Name
101	John Bell
101	Jonathan Bell
102	
104	Claire Stevens
107	Pat Norton

Students	
Snum	Name
101	John Bell
101	Jonathan Bell
104	Claire Stevens
107	Pat Norton

Snum	Name
101	John Bell
101	Jonathan Bell
104	Claire Stevens
107	Pat Norton



SOLUTION DIRECTION

REPAIRS AND CONSISTENT ANSWERS

How? There can be huge numbers of possible repairs!

- Transform query!

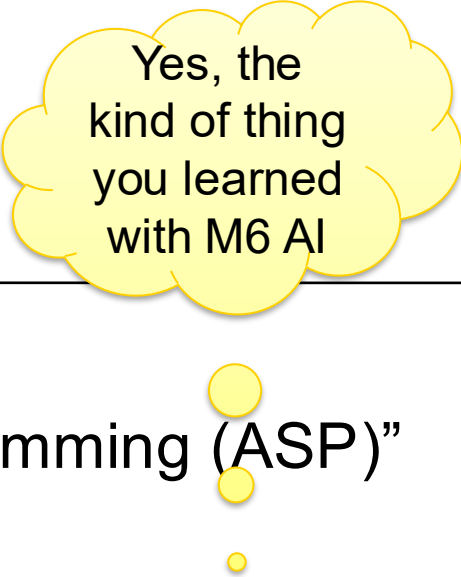
- **SELECT * FROM Students**

i.e., in our formal notation: $Students(x,y)$

- Constraint: $Students(x,y) \wedge \neg \exists z (Students(x,z) \wedge z \neq y)$

```
SELECT *  
FROM Students s1  
WHERE NOT EXISTS  
  (SELECT * FROM Students s2  
   WHERE s1.Snum = s2.Snum  
        AND s1.Name != s2.Name)
```

SOLUTION DIRECTION REPAIRS AND CONSISTENT ANSWERS



Yes, the
kind of thing
you learned
with M6 AI

Rewriting queries for CQA is not trivial

➤ Another approach is “Answer-Set Programming (ASP)”

Answer-set program: Declarative program that specifies all possible repairs (executable specification)

- Chp III-1 section 3.1 describes this quite theoretically;
I developed a hopefully more insightful approach

Main idea is to determine *all possible ways* what a repaired table could look like ...

REMEMBER OUR EXAMPLE OF A VIOLATION OF SNUM → NAME

Students

Snum	Name	Snum	Name
101	John Bell	101	John Bell
101	Jonathan Bell	101	Jonathan Bell
104	Claire Stevens	104	Claire Stevens
107	Pat Norton	107	Pat Norton

Snum	Name
101	John Bell
102	
101	Jonathan Bell
104	Claire Stevens
107	Pat Norton

Snum	Name
101	John Bell
101	Jonathan Bell
102	
104	Claire Stevens
107	Pat Norton

Four possible minimal repairs

- Idea behind a repair is that it corrects a mistake someone or something made
- By mistake entered person twice → DEL repair
 - By mistake assigned same Snum → UPD repair
 - (By mistake gave wrong name → UPD repair)

REMEMBER OUR EXAMPLE OF A VIOLATION OF SNUM → NAME

So, 4 possible repaired tables

- We could do CQA by executing our query on all 4 possible repaired tables and taken intersection (= answers consistently given in all possible repaired tables)

Snum	Name
102	John Bell
101	Jonathan Bell
104	Claire Stevens
107	Pat Norton



Snum	Name
101	John Bell
104	Claire Stevens
107	Pat Norton

Snum	Name
101	Jonathan Bell
104	Claire Stevens
107	Pat Norton

Snum	Name
101	John Bell
102	Jonathan Bell
104	Claire Stevens
107	Pat Norton

WHAT IF WE HAD TWO VIOLATIONS OF SNUM → NAME?

Students

Two violations of FD Snum → Name

- First had 4 possible repairs
- Second has 4 possible repairs too for the same reasons
- Behind it are two ***independent*** mistakes that we are trying to repair

Snum	Name
101	John Bell
101	Jonathan Bell
104	Claire Stevens
104	Pat Norton

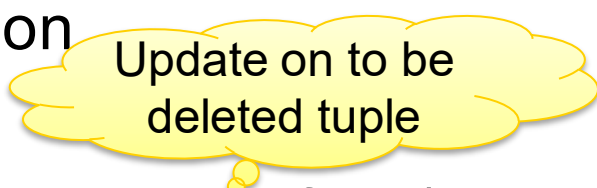
➤ So, $4 * 4 = 16$ possible repaired tables!

❖ Still, our computers are fast and we can handle a lot of data, so can we generate this automatically?

CAN WE AUTOMATICALLY GENERATE ALL POSSIBLE REPAIRED TABLES?

Of course

- Determine violations of the FD (violation=pair of tuples)
- Generate possible repairs per violation
- Create repair combinations
- Clean repair combinations (duplicates and conflicts)
- Calculate repair cost (e.g., update=1, delete=3)
- Apply repair combinations (output=repaired tables)
- Validate & filter: repaired table should satisfy FD
- Find cheapest valid repaired table(s)



Update on to be deleted tuple

CAN WE AUTOMATICALLY GENERATE ALL POSSIBLE REPAIRED TABLES?

DQ Lab 2 contains a Python notebook that does this

- 20-row 11-column table on music albums
- Functional dependency: album, year $\rightarrow$ genre
- 3 Violations $\rightarrow 4^3 = 64$ possible repaired tables
- Fill in few small bits of code and answer quiz questions

- Outcome is that cheapest repair has cost 2
 - ... so with only 2 updates we could repair 3 violations
 - ... most likely situation is that only 2 minor mistakes were made to cause these 3 data quality problems

I hope your curiosity about how this is possible may make this lab exercise more interesting for you 😊

Given Students(X,Y) predicate
Specify repaired table as predicate Students_(X,Y,A)
The A=t** facts represent rows in the final repaired table

ANSWER-SET PROGRAMMING

Answer-set program: Declarative program that specifies all possible repairs (executable specification)

- Chp III-1 section 3.1 describes this quite theoretically;
My Python-based approach is imperative

Not exam material

Declarative approach Chp III-1 section 3.1

- Use a logic programming language to specify declaratively all possible repaired tables
- Reasoning engine can use this to answer CQA or other questions about all possible repaired tables without necessarily exhaustively generating them

Yes, the kind of thing you learned with M6 AI

ANSWER SET PROGRAMMING

Answer-set program: Declarative program that specifies all possible repairs (executable specification)

- Define new table contents
 - $\text{Students_}(X,Y,A) \leftarrow \dots \text{Students}(X,Y) \dots$
- with annotations A being
 - **t**: repair by insertion of the tuple in the new table
 - **f**: repair by deletion of the tuple from the new table
 - **t***: tuple was in the original table or inserted in the repair process
 - **t****: tuple belongs to final contents of the new table

ANSWER SET PROGRAMMING

EXAMPLE

New table: Students_

Answer-set program

1. $\text{Students_}(x,y,\mathbf{f}) \vee \text{Students_}(x,z,\mathbf{f}) \leftarrow$
 $\text{Students_}(x,y,\mathbf{t^*}) \wedge \text{Students_}(x,z,\mathbf{t^*})$
 $\wedge y \neq z \wedge x \neq \text{null} \wedge y \neq \text{null} \wedge z \neq \text{null}$
2. $\text{Students_}(x,y,\mathbf{t^*}) \leftarrow \text{Students}(x,y).$
3. $\text{Students_}(x,y,\mathbf{t^*}) \leftarrow \text{Students_}(x,y,\mathbf{t}).$
4. $\text{Students_}(x,y,\mathbf{t^{**}}) \leftarrow \text{Students_}(x,y,\mathbf{t^*}) \wedge$
 $\neg \text{Students_}(x,y,\mathbf{f})$
5. $\leftarrow \text{Students_}(x,y,\mathbf{t}) \wedge \text{Students_}(x,y,\mathbf{f})$

ANSWET SET PROGRAMMING

EXAMPLE

If there are referential integrity constraints (e.g., Enrollment table), then more rules are needed to also insert/delete there.

Not exam material

Explanation of clauses on previous slide

1. Rule to enforce the functional dependency
If there are two tuples with the same Snum but different Names, then propose to delete either of those tuples
2. Ensure that original tuples are possibly in the repaired table
3. Ensure that tuples proposed to be inserted are possibly in the repaired table
4. Collect all tuples that can be in the final repaired table
5. Program constraint the discards solutions where a tuple is both proposed to be inserted and deleted

Not exam
material

WHAT HAPPENS TO OUR EXAMPLE?

Rule 2

Students_(101,John Bell,t*)
Students_(101,Jonathan Bell,t*)
Students_(104,Claire Stevens,t*)
Students_(107,Pat Norton,t*)

Rule 1

Students_(101, John Bell,f) v Students_(101,Jonathan Bell,f)

Rule 4

Students_(104,Claire Stevens,t**)
Students_(107,Pat Norton,t**)
Students_(101,Jonathan Bell,t**) v Students_(101, John Bell,t**)

Students	
Snum	Name
101	John Bell
101	Jonathan Bell
104	Claire Stevens
107	Pat Norton

Only one of these two
tuples in new table (2
possible repairs)

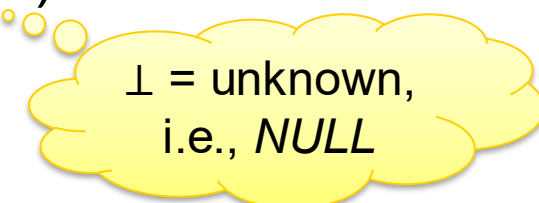
ACTIVE INTEGRITY CONSTRAINTS

Instead of reasoning about *possible* repairs, one can also ask the user to specify the *intended* repair

- $\forall x \forall y (\text{Enrollment}(x,y) \wedge \neg \exists z \text{ Students}(x,z) \supset \neg \text{Enrollment}(x,y)$

Approach could also be open to specifying several possible repairs

- $\forall x \forall y (\text{Enrollment}(x,y) \wedge \neg \exists z \text{ Students}(x,z) \supset \neg \text{Enrollment}(x,y) \vee +\text{Students}(x,\perp)$



$\perp$ = unknown,
i.e., *NULL*

VIRTUAL DATA INTEGRATION

Unified view of data in multiple sources through *mediator*

- Define global schema
- Define mappings between global and source schemas
- Queries on global schema automatically rewritten to execute on source schema(s)
- Use CQA or repair approach to deal with inconsistencies between the sources

VIRTUAL DATA INTEGRATION: MAPPINGS

StuPs		StuCS		
Snum	Name	Snum	Name	Admission
101	John Bell	104	Alex Pelt	2011
104	Claire Stevens	121	Ana Jones	2012
107	Pat Norton	137	Lisa Reeves	2012

Global schema: StuUniv(StuNum,StuName,Adm,Dept)
Mappings

- $\text{StuPs}(x,y) \leftarrow \text{StuUniv}(x,y,z,\text{"Psychology"})$
- $\text{StuCS}(x,y,z) \leftarrow \text{StuUniv}(x,y,z,\text{"Computer Science"})$

Local-as-view (LAV)

VIRTUAL DATA INTEGRATION: QUERYING

Querying based on CQA (minimal legal instances)

Examples

- $Q_3(x) : \exists y \exists z \exists w \text{ StuUniv}(x, y, z, w)$

- Answer $Q_3(x) = \{101, 104, 107, 121, 137\}$

- $Q_4(x, z) : \exists y \exists w \text{ StuUniv}(x, y, z, w)$

- Answer $Q_4(x, z) = \{(104, 2011), (121, 2012), (137, 2012)\}$

More about missing values later

VIRTUAL DATA INTEGRATION: MISSING VALUES

What to do with the missing values (*NULLs*)?

- Let the system 'invent' a possible value

$\text{StuPs}(x,y) \leftarrow \text{StuUniv}(x,y,z, \text{"Psychology"}), F_1(x,y,z)$

$\text{StuCS}(x,y,z) \leftarrow \text{StuUniv}(x,y,z, \text{"Computer Science"})$

$\text{dom}(\mathbf{X})$.

$F_1(x,y,z) \leftarrow \text{StuPs}(x,y), \text{dom}(z), \text{choice}((x,y),z)$

$\text{dom}(z)$ fixes a
domain for z

Invent a value for
 z that satisfies FD
 $X,Y \rightarrow Z$

VIRTUAL DATA INTEGRATION: GLOBAL CONSTRAINTS

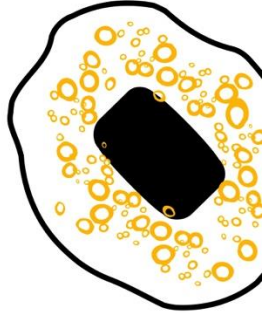
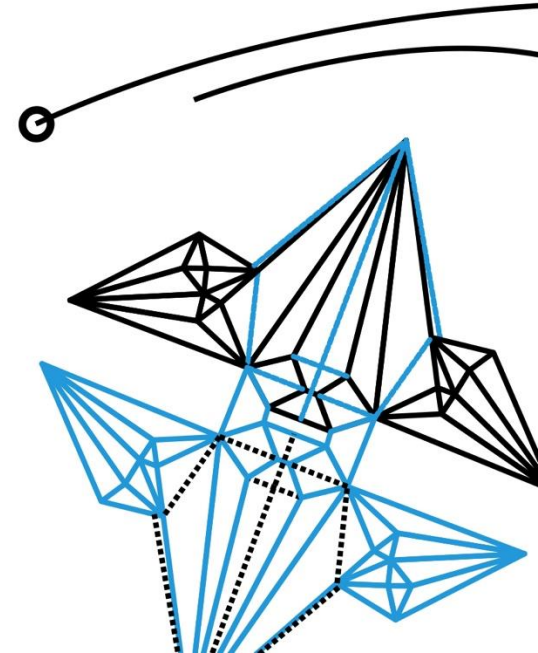
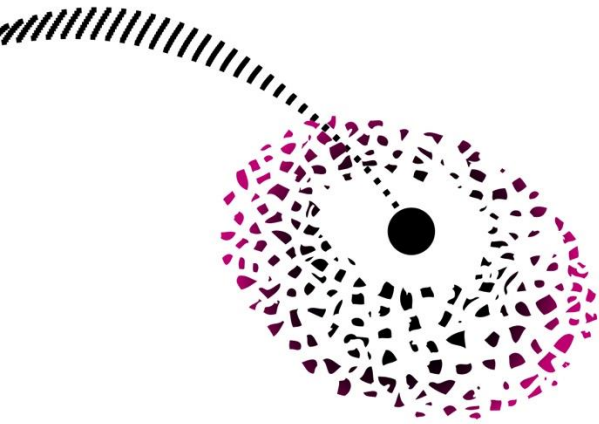
StuPs		StuCS		
Snum	Name	Snum	Name	Admission
101	John Bell	104	Alex Pelt	2011
104	Claire Stevens	121	Ana Jones	2012
107	Pat Norton	137	Lisa Reeves	2012

We'd like to impose the Snum is globally unique

- It is unique in both departments, but not globally
- Approach in previous slides: repair by deletion
- But we could use answer set programming or active constraints to suggestion insertions
- Or we let the mediator 'invent' new globally unique IDs

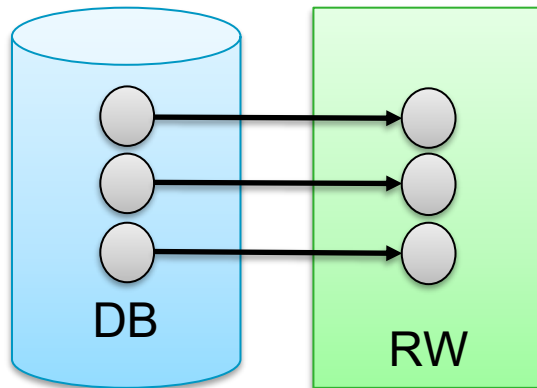
FINDING DUPLICATES = RECORD MATCHING

ENTITY RESOLUTION = MATCHING + RESOLVING THEM

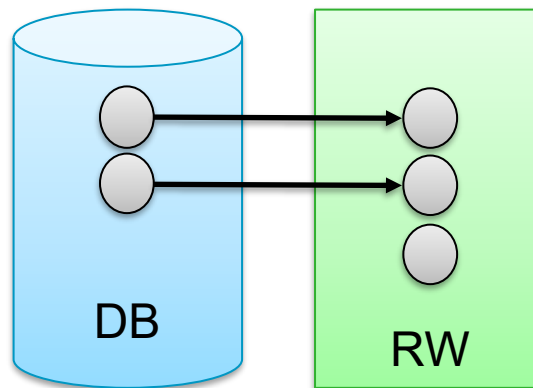


A DATABASE REPRESENTS THE REAL WORLD

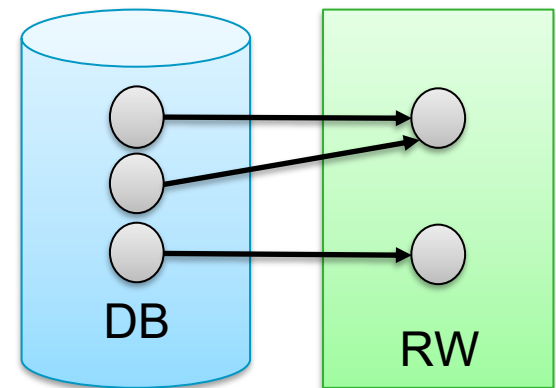
FIG 1 ON PAGE 237 (ADAPTED)



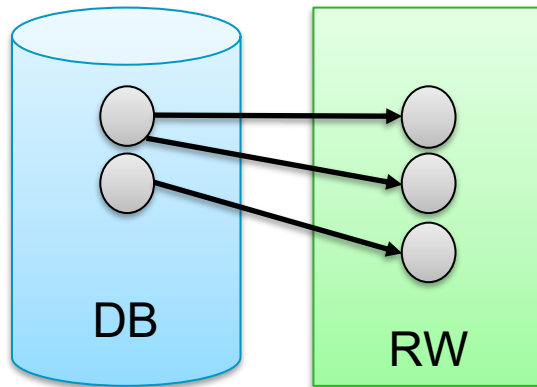
Proper representation



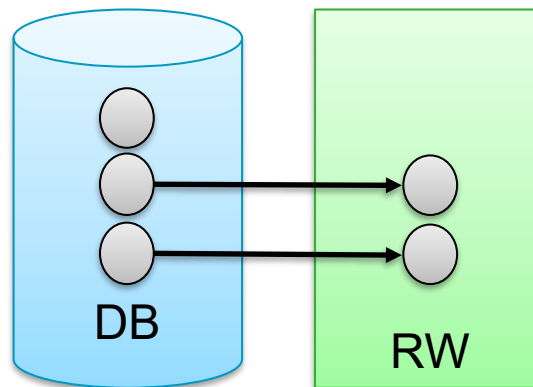
Incomplete representation



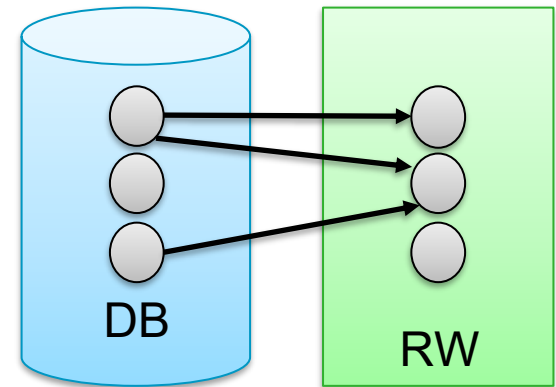
Duplicate



Ambiguous representation



Incorrect representation



It can become quite complex in practice

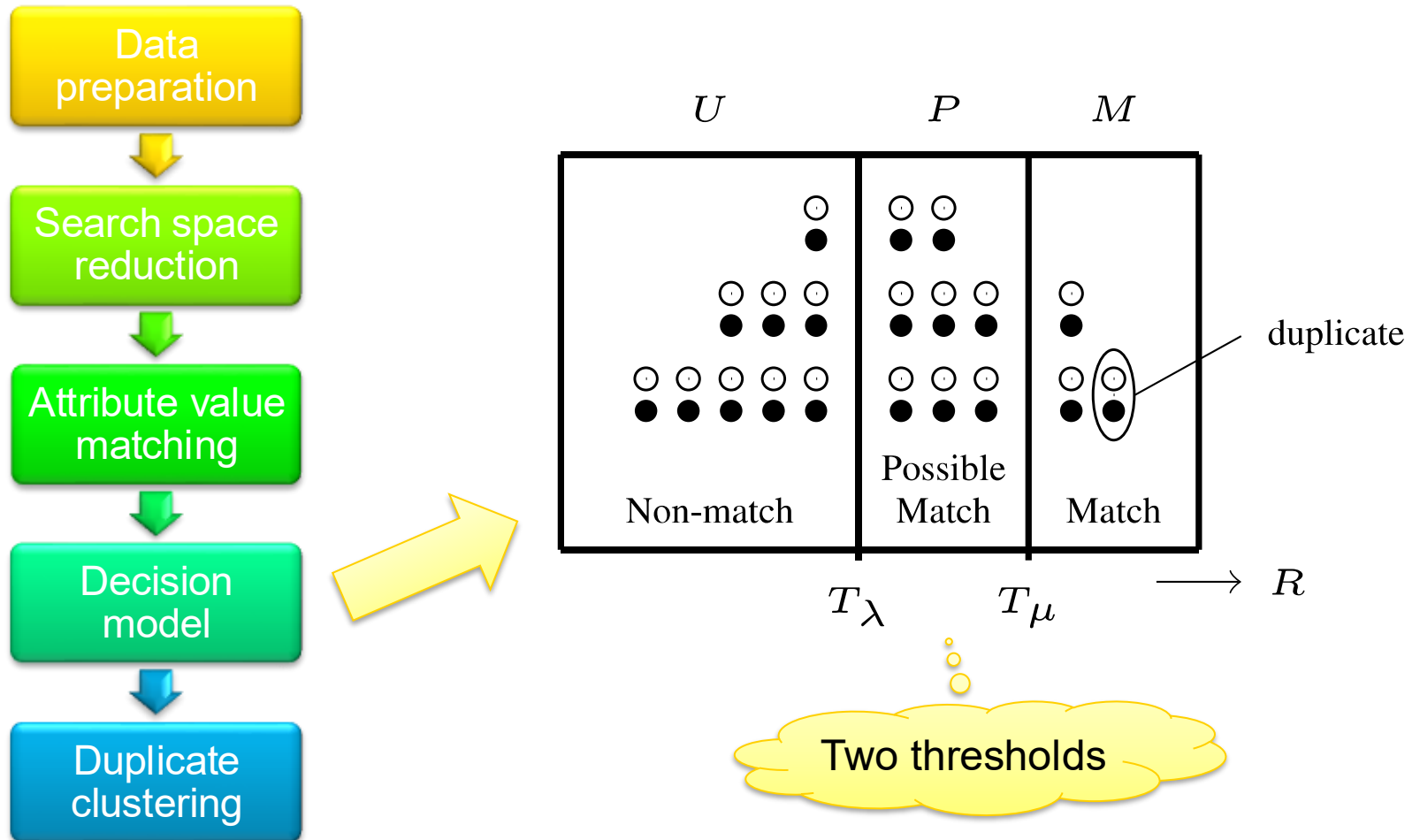
COMPONENTS OF ENTITY RESOLUTION



Common
task in NLP

- Entity reference extraction
(from unstructured information)
- Entity reference preparation
(profiling, standardization, cleaning, etc. before ER)
- **Entity reference resolution**
Comparison of pairs: same RW entity or not?
Identity/matching rules measure degree of similarity
- Entity identity information management
(store and accumulate identity info over time in entity
identity structures (EIS))
- Entity analytics
(investigating associates, e.g. families, subsidiaries)

RECORD MATCHING PROCESS



ENTITY REFERENCE PREPARATION

- Data profiling
- Parsing, e.g., “John A. Doe” → “John” / “A” / “Doe”
 - Not easy: “Boulevard 1945” is a street in Enschede, not an address with number 1945 on street Boulevard!
 - “Maurice Van Keulen” and “Jan Willem Alfenaar”
“Van” belongs to lastname; “Willem” belongs to firstname
- Standardization, e.g., “123 Oak St.” → “123 Oak Street”
- Encoding, e.g., LATIN1 → UTF8 ... “f?r” → “für”
- Conversion, e.g., “123,59” (string) → 123.59 (number)
- Enhancement: adding info not in original reference
e.g., add log/lat coordinates to an address

DATA PROFILING

Often used: pattern frequency analysis

- Replace digits with '9', capitals with 'A', lowercase 'a'
 - “818-453.1558” → “999-999.9999”
 - Then count occurrences of all the patterns
- Find values of often occurring types (email, phone, date, creditcard number)

ENTITY REFERENCE RESOLUTION

RECORD MATCHING, RECORD LINKAGE, ENTITY MATCHING

Exact matching vs. probabilistic or fuzzy matching

- “JOHN DOE” $\neq$ “John Doe” $\neq$ “J. Doe” $\neq$ “John Doe”

Fuzzy matching with similarity functions

- Numerical: allowable difference (numbers, times, dates)
- Syntactic: see next slides
- Semantic: “Jim” and “James” semantically equivalent
- Phonetic: “Karen” and “Caryn”
- Hybrid: combinations of the above

Per attribute and then weighted combination for obtaining record similarity

JACCARD COEFFICIENT

Formula: $Jaccard(P, Q) = \frac{|P \cap Q|}{|P \cup Q|}$

- Essence fraction of sharing between P and Q

Example

- $P = \{\text{"John", "Doe"}\}$, $Q = \{\text{"Sean", "Doe"}\}$
 $Jaccard(P, Q) = 1/3 = 0.333$
- $P = \{\text{"J", "o", "h", "n", "D", "o", "e"}\} \rightarrow \{\text{"e", "h", "n", "o", "D", "J"}\}$
 $Q = \{\text{"S", "e", "a", "n", "D", "o", "e"}\} \rightarrow \{\text{"a", "e", "n", "o", "D", "S"}\}$
 $Jaccard(P, Q) = 4/8 = 0.5$

Not sensitive to word order, but $Jaccard(\text{"Sara", "Saar"}) = 1$

LEVENSHTEIN DISTANCE

Formula: $N_Levenshtein = 1 - \frac{L(A,B)}{\max(\text{length}(A), \text{length}(B))}$

- $L(A,B)$: number of manipulations to turn A into B
- Example: JIM → JAMES needs 3 manipulations
 - Update I → A, insert E, insert S
 - So, Normalized Levenshtein = $1 - 3/5 = 0.4$

Widely used, but sensitive to pre/postfixes and abbreviations

- “Professor John Doe” and “John Doe”
- “Peter J Miller” and “Peter John Miller”

JARO AND JARO-WINKLER DISTANCE

Formula: $J(A, B) = W_1 \frac{C}{L_A} + W_2 \frac{C}{L_B} + W_3 \frac{(C-T)}{C}$

- $W_1 + W_2 + W_3 = 1$
- C: Number of common characters
- T: Number of transpositions (below: LE→EL)

Example

- $J(SHAKLER, SHAKEL) = \frac{1}{3} \frac{6}{7} + \frac{1}{3} \frac{6}{6} + \frac{1}{3} \frac{(6-1)}{6} = 0.897$

Jaro-Winkler is modification

- N: number of first four characters that agree position
- $W(A, B) = J(A, B) + 0.1 \cdot N \cdot (1.0 - J(A, B))$
- Example: $0.897 + 0.1 \cdot 4 \cdot (1.0 - 0.897) = 0.938$

OTHER DISTANCE METRICS

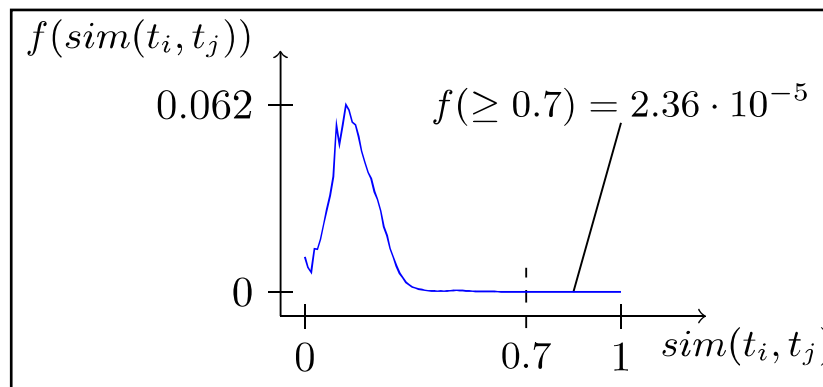
- Smith-Waterman distance
 - Longest common subexpression
 - Example: “Professor John Doe” and “John R Doe” → “John Doe”, so 8. Then normalized often
- Q-gram distance (also called n-gram)
 - Idea: create substrings of length Q
 - “John Doe” → [“Joh”, “ohn”, “hn “, “n D”, “ Do”, “Doe”]
 - Then use Jaccard or or Jaro-Winkler or longest common sequence divided by longest sequence

CONVERTING SCORES INTO PROBABILITIES

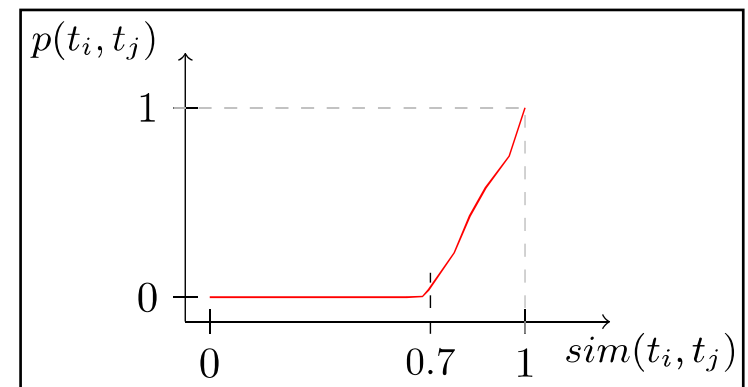
Common is to normalize

- Turns score into value between 0 and 1
- But this is not a probability
(doesn't behave like a probability)

Better: *sim2p-mapping* (supervised)



Relative frequency of similarity scores

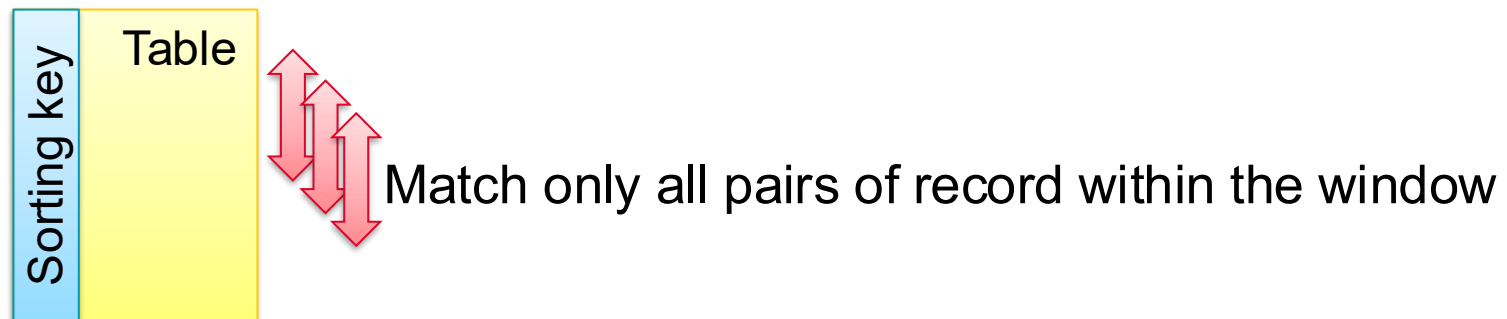


Sim2p-mapping

EFFICIENCY OF ENTITY MATCHING

Problem: entity matching needs to compare every record with every record, so complexity $O(n^2)$

- Sorted neighbourhood method (SNB)
 - Avoid matching those that are not likely to match
 - Sort on a sorting key; match only in a sliding window



Sorting key crucial: Hard to avoid missing matches

MEASURING AND ANALYZING MATCHING RESULTS (1)

Result of matching set of records S is a partitioning

- Partition of S is set of non-empty subsets
- Union of subsets is S
- Intersection of each pair of subsets is empty
- Subsets are often called *clusters*

MEASURING AND ANALYZING MATCHING RESULTS (2)

We need a sample for which we know the correct linking

Measuring

- T is partition obtained by *correct* linking
- X is partition obtained by our matching algorithm
- V is partition formed by all non-empty intersections between true clusters (T) and process clusters (X)
- Talburt-Wang Index: $TWI = \frac{\sqrt{|T| \cdot |X|}}{|V|}$
- E: set of pairs of records in T; L: set of pairs in X
- $\sim E$: pairs in S not in E; $\sim L$: pairs in S not in L
- False positive rate: $FPR = |L \cap \sim E| / |\sim E|$
- False negative rate: $FNR = |\sim L \cap E| / |E|$

ENTITY IDENTITY INFORMATION MANAGEMENT

We strive for

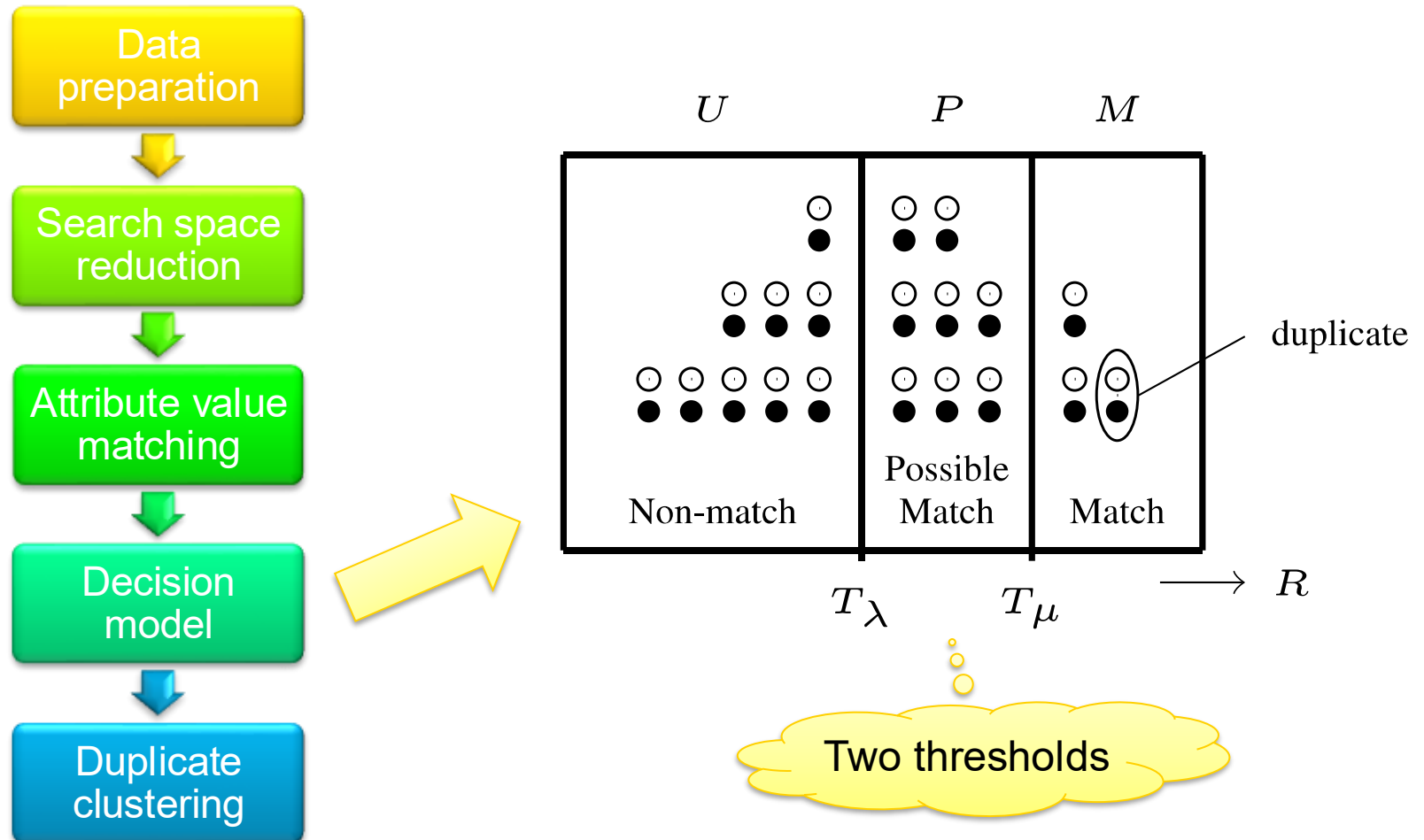
- Each RW entity has exactly one occurrence in the data
- Distinct RW entities have distinct occurrences

We have a cluster of records (duplicates)

- Survivor record EIS: keep most complete one
- Attribute-based EIS: surrogate record created with a most representative value for each attribute
 - Variant: keep all possible values
- Record-based EIS: keep all possible records

RECORD MATCHING PROCESS

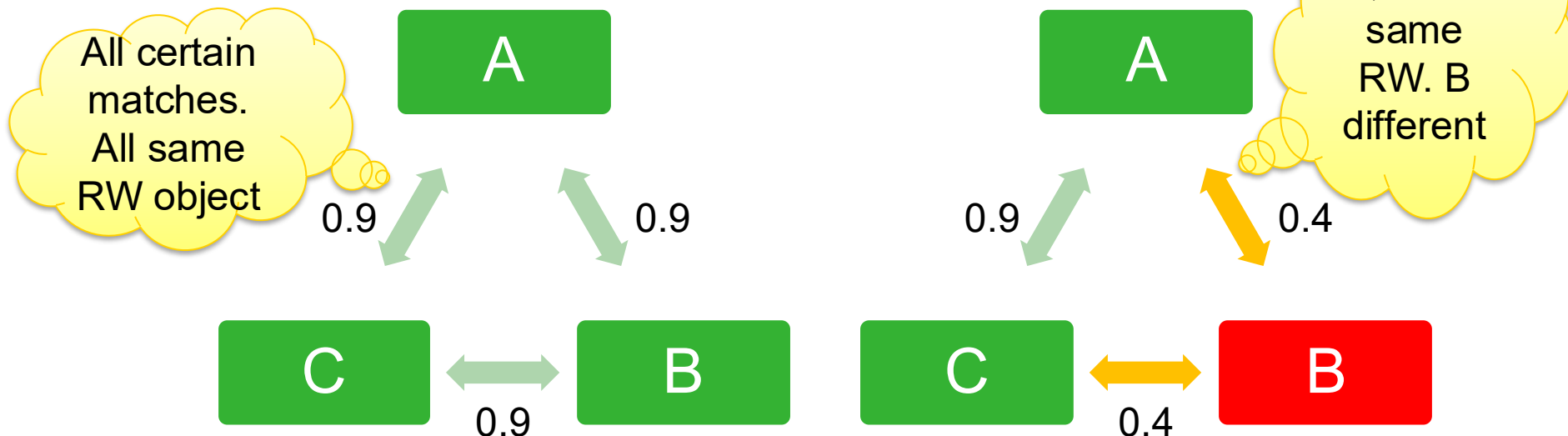
CLEANING DUPLICATES



CLUSTERING (1)

Result of matching is a set of matching record pairs

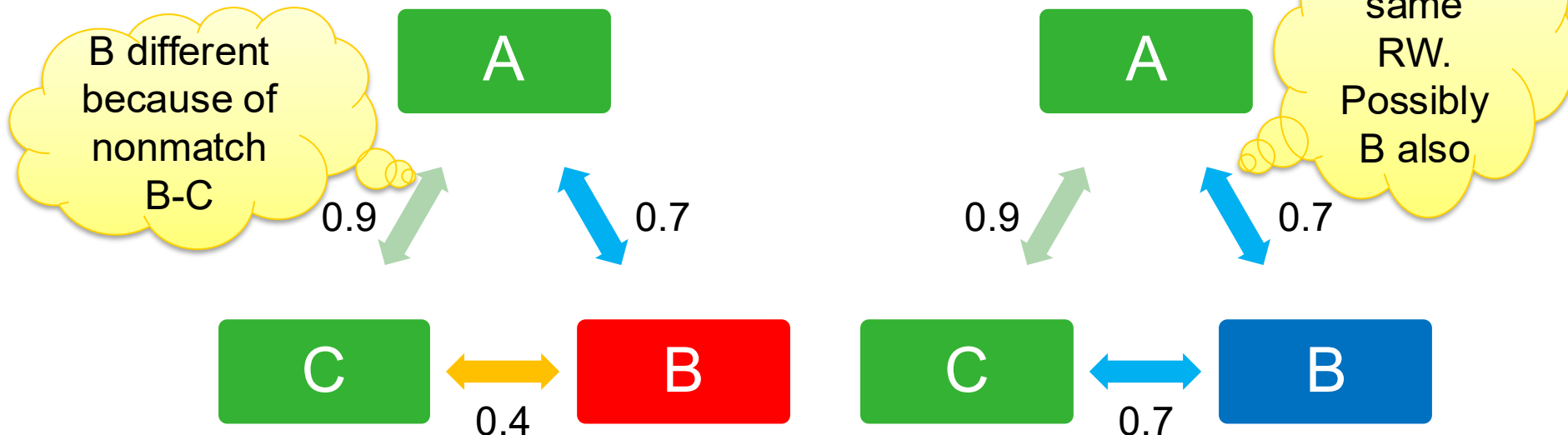
- Equality is transitive, so we cluster by transitive closure
- Let's take thresholds $T_\lambda = 0.5$ and $T_\mu = 0.8$



CLUSTERING (2)

Result of matching is a set of matching record pairs

- Equality is transitive, so we cluster by transitive closure
- Let's take thresholds $T_\lambda = 0.5$ and $T_\mu = 0.8$



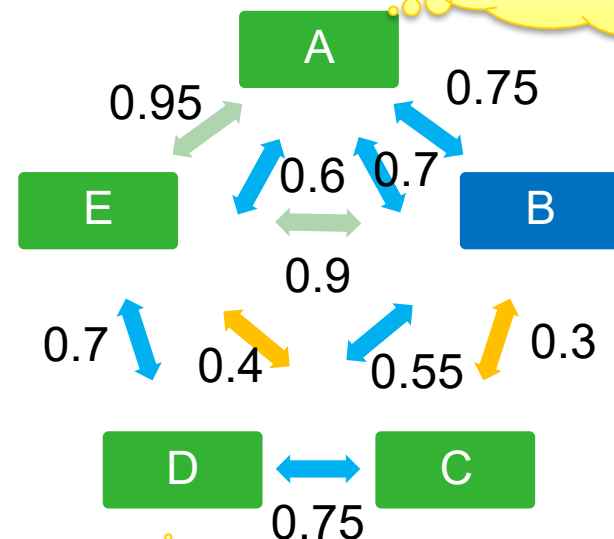
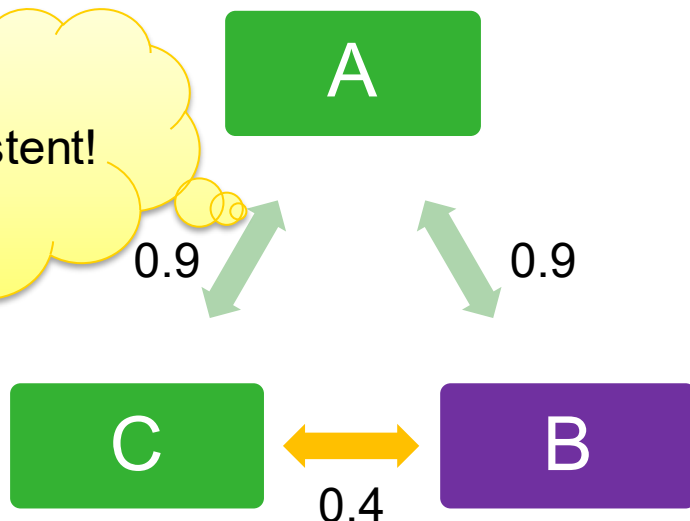
CLUSTERING (3)

Result of matching is a set of matching record pairs

- Equality is transitive, so we cluster by transitive closure
- Let's take thresholds $T_\lambda = 0.5$ and $T_\mu = 0.8$

Complicated

Inconsistent!



Partitionings: {ABE,CD}, {ABDE,C}, {ABE,C,D}

RESOLUTION OF POSSIBLE PARTITIONINGS

Partitionings: { {ABE,CD}, {ABDE,C}, {ABE,C,D} }

How to proceed now?

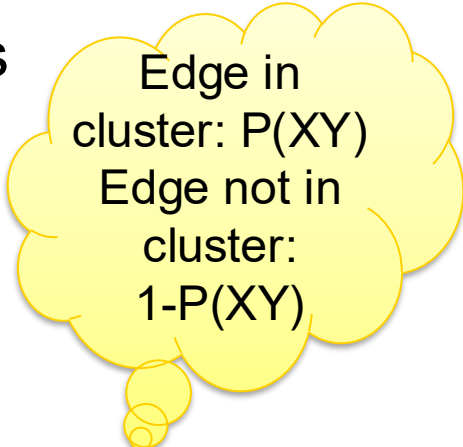
1. Have a human determine which one it is

- Decide D being same RWO with ABE
- Decide C and D being same RW
- Both cannot be true at the same time

2. Ranking and take most probable one

- $P(\{ABE,CD\}) = P(AB) * (1-P(AC)) * (1-P(AD)) * P(AE) * (1-P(BC)) * (1-P(BD)) * P(BE) * P(CD) * (1-P(CE)) * (1-P(DE))$

3. *There is a third main option which I will adress later*



Edge in
cluster: $P(XY)$
Edge not in
cluster:
 $1-P(XY)$

HOW TO HANDLE MERGE INCONSISTENCIES

We detected a duplicate:
How to merge the records?

- Determine an ID (e.g., cluster ID)
- Any foreign keys in other tables referring to this ID, need updating
- Determine value for other attributes
 - Choose most likely value
 - or combination formula

Example: sales on product $\Rightarrow$ SUM(sales)

Students	
Snum	Name
101	John Bell
101	Jonathan Bell
104	Claire Stevens
107	Pat Norton
111	Pat Norton

➤ **Goal:** One record with one value for each attribute

RESOLUTION OF POSSIBLE PARTITIONINGS

Partitionings: { {ABE,CD}, {ABDE,C}, {ABE,C,D} }

How to proceed now?

1. Have a human determine which one it is
2. Ranking and take most probable one
3. This is a form of uncertainty ...

... are there no uncertain databases?

Databases where you can *store possibilities* and that, when queried, produces *possible answers*?

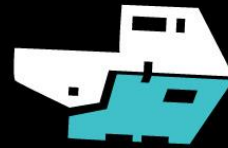
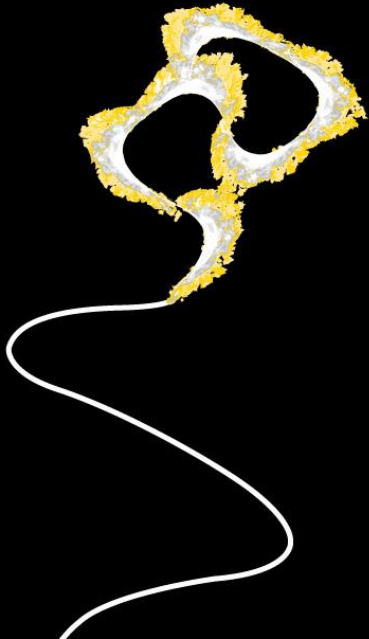
There are!
(in research)

UNIVERSITEIT TWENTE.

Intermezzo

(only slides examined on
main concepts and understanding)

PROBABILISTIC DATABASES TO THE RESCUE



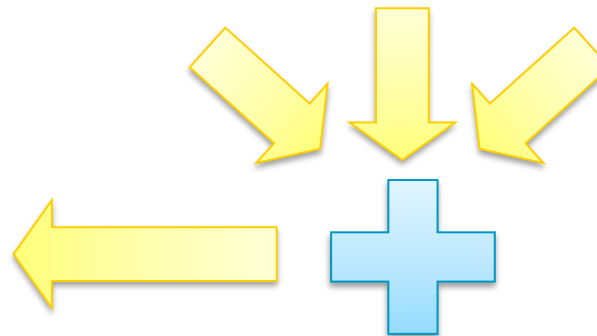
THREE SOURCES TO COMBINE

Car brand	Sales
B.M.W.	25
Mercedes	32
Renault	10

Car brand	Sales
BMW	72
Mercedes-Benz	39
Renault	20

Car brand	Sales
Bayerische Motoren Werke	8
Mercedes	35
Renault	15

Car brand	Sales
B.M.W.	25
Bayerische Motoren Werke	8
BMW	72
Mercedes	67
Mercedes-Benz	39
Renault	45



Keulen, M. (2012) *Managing Uncertainty: The Road Towards Better Data Interoperability*. IT - Information Technology, 54 (3). pp. 138-146. ISSN 1611-2776

... AND THE PROBLEM OF SEMANTIC DUPLICATES

Car brand	Sales
B.M.W.	25
Bayerische Motoren Werke	8
BMW	72
Mercedes	67
Mercedes-Benz	39
Renault	45

Preferred customers ...

```
SELECT SUM(Sales)
FROM CarSales
WHERE Sales>100
```



0

'No preferred customers'

MOST DATA QUALITY PROBLEMS CAN BE MODELED AS UNCERTAINTY IN DATA

Run some duplicate
detection tool

	Car brand	Sales
1	B.M.W.	25
2	Bayerische Motoren Werke	8
3	BMW	72
4	Mercedes	67
5	Mercedes-Benz	39
6	Renault	45
	Mercedes	106
	Mercedes-Benz	106

X=0

X=0

X=1

Y=0

X=1

Y=1

X=0	4 and 5 different	0.2
X=1	4 and 5 the same	0.8
Y=0	"Mercedes" correct name	0.5
Y=1	"Mercedes-Benz" correct name	0.5

B.M.W. / BMW / Bayerische Motoren Werke analogously

WHAT I HAVE NOW IS A PROBABILISTIC DATABASE

- Looks like ordinary database
- Several “possible” answers or approximate answers to queries
- Important: Scalability (big data!)

Sales of “preferred customers”

- `SELECT SUM(sales)`
`FROM carsales`
`WHERE sales ≥ 100`

SUM(sales)	P
0	14%
105	6%
106	56%
211	24%

QUERYING AND RELIABILITY ASSESSMENT

Sales of “preferred customers”

- `SELECT SUM(sales)`
`FROM carsales`
`WHERE sales ≥ 100`

- Answer: 106

 Risk of substantially wrong answer

- Risk = Probability * Impact
- Analyst only bothered with problems that matter

SUM(sales)	P
0	14%
105	6%
106	56%
211	24%

Second most likely answer at 24% with impact factor 2 in sales (211 vs 106)

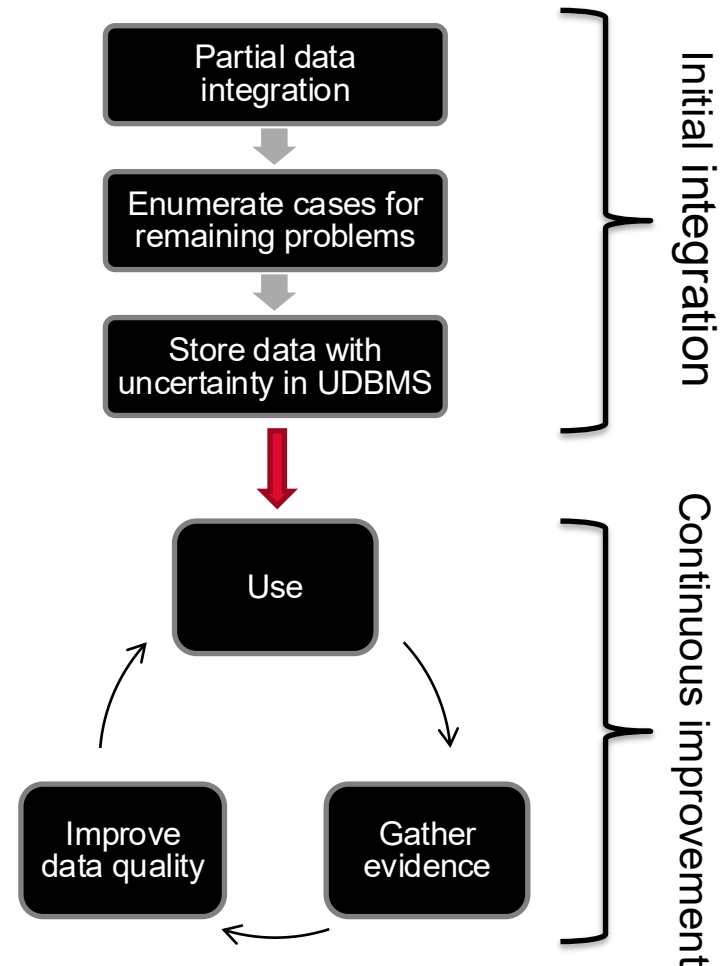
PROBABILISTIC DATA INTEGRATION

LET'S SIMPLY NOT SOLVE THE HARDEST 10% UNCERTAIN CASES! (RIGHTAWAY)

Let's go for an initial integration that can readily and meaningfully be used

“Good is good enough” for meaningful use in many applications
(achievable 10x earlier)

Let it improve during use



CONTINUOUS DATA QUALITY IMPROVEMENT

EXAMPLE AMBIGUOUS CASE: "BADMAN BEGINS / 2005"

TV Guide

IMDB

A

Batman begins / 2005
Action, Crime, Fantasy

B

Batman begins / 2005
Action, Adventure,
Crime, Thriller

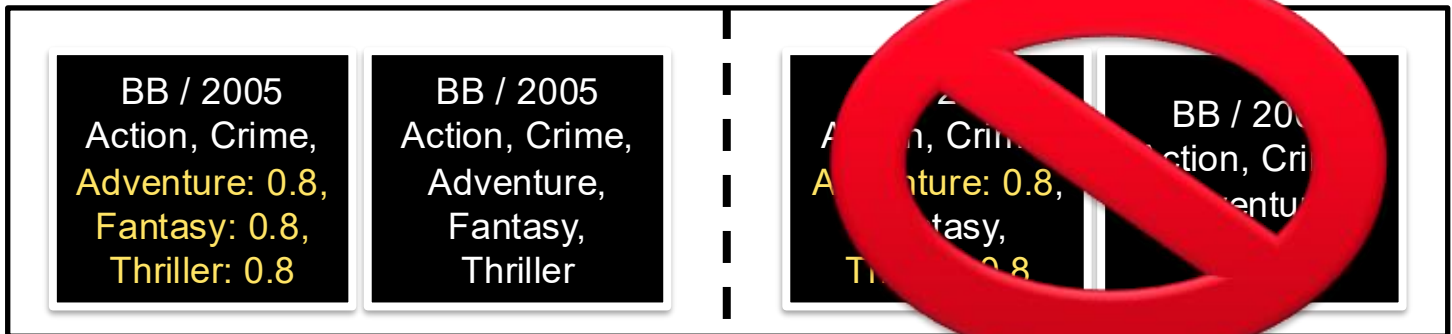
C

Batman begins / 2005
Action, Adventure,
Crime, Fantasy, Thriller

Ambiguity: $A=B \neq C$ or $A=C \neq B$

0.4 → 1.0

0.6



Plot: This game is
largely based on the
movie ...

Query: How many fantasy movies?

1 (0.68) or 2 (0.32)!

1 (0.2) or 2 (0.8)

UNIVERSITEIT TWENTE.

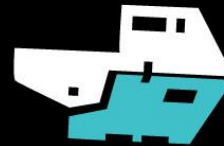
End of intermezzo
(only slides examined on
main concepts and understanding)

**PROBABILISTIC DATABASES
TO THE RESCUE**

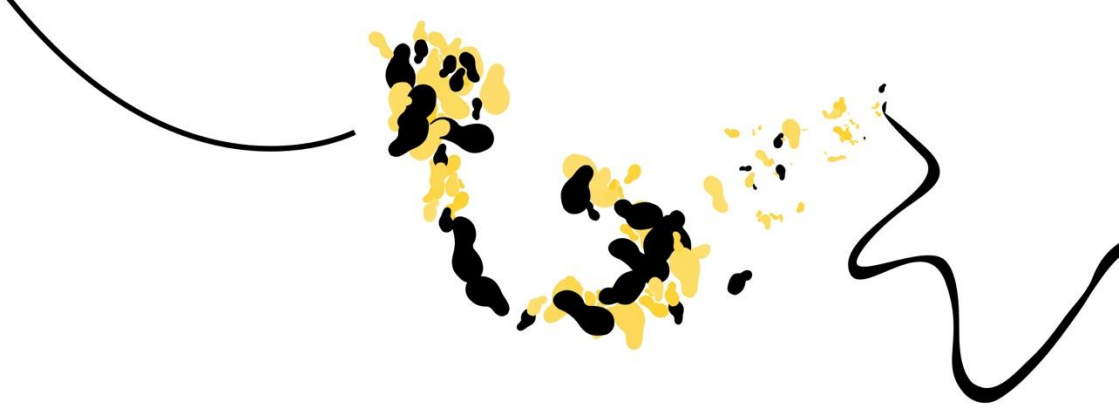


DuBio

UT research
prototype

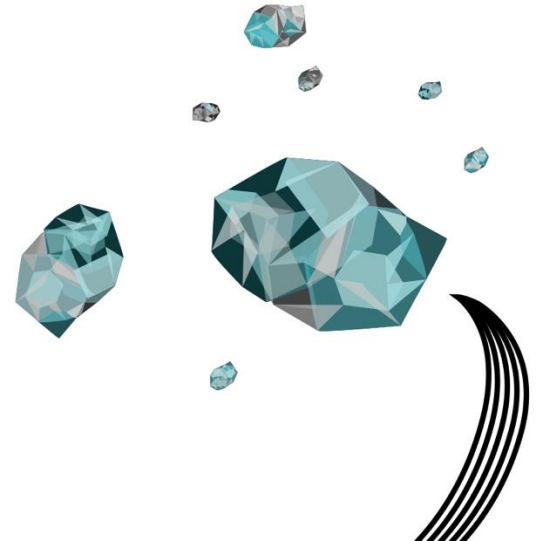
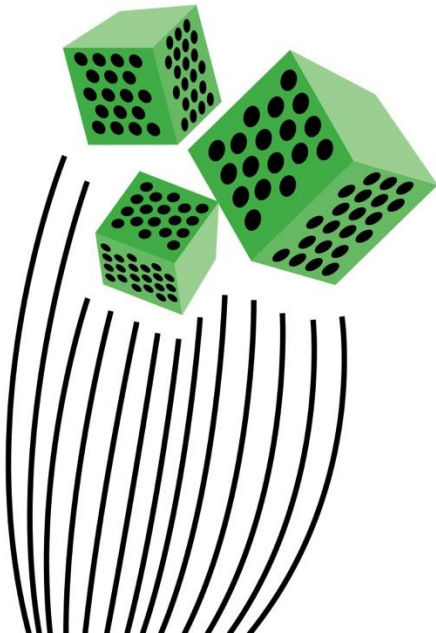


UNIVERSITEIT TWENTE.



ANOMALY DETECTION

NOT IN THE BOOK



ANOMALY DETECTION / OUTLIER DETECTION

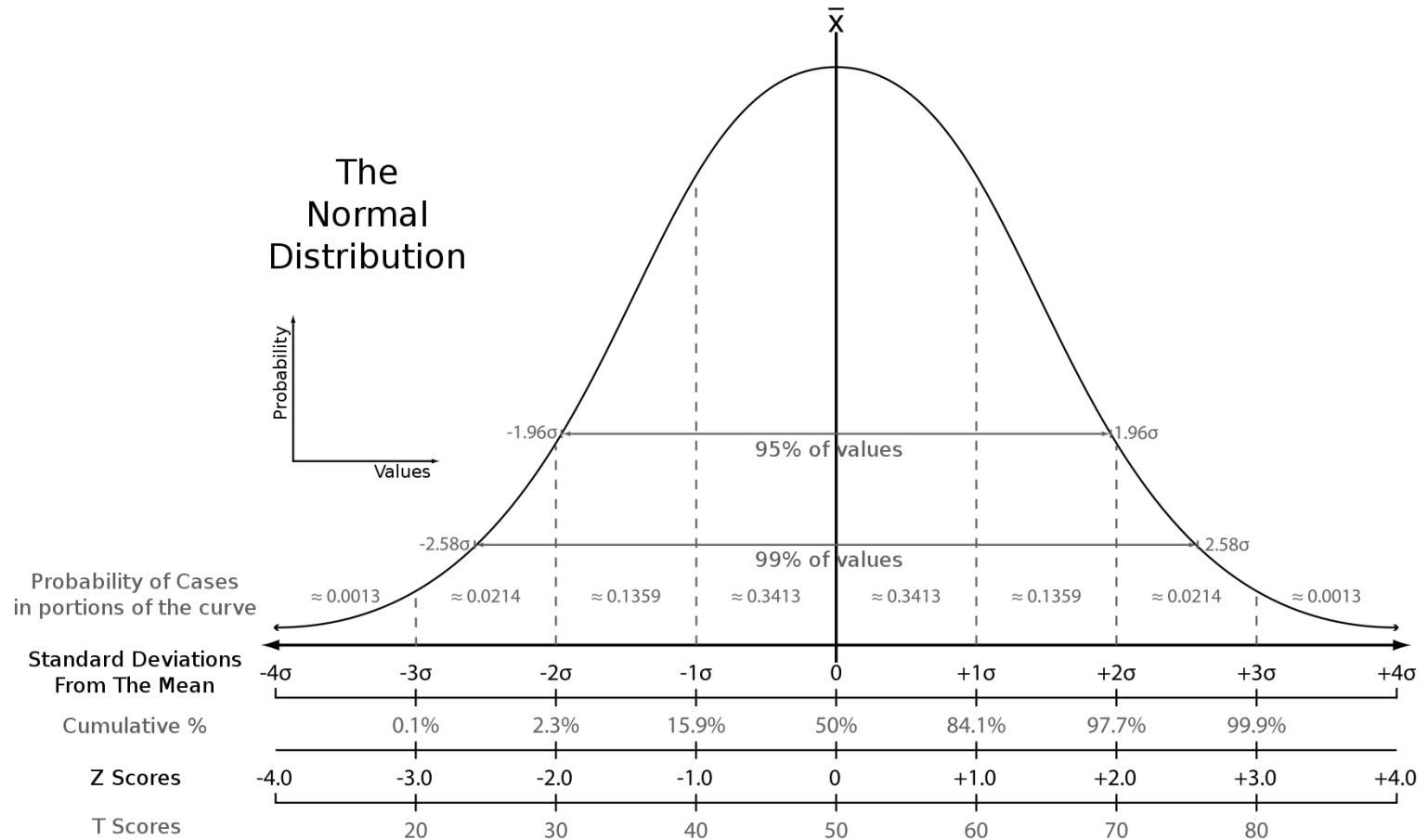
Define or obtain model of “normal behaviour”

- Measure likelihood of record/value to be generated by this model
- Outliers are rare and/or inconsistent with the rest and/or far away from the rest according to some distance measure

Categorisation of approaches

- Supervised
- Semi-supervised
- Unsupervised (most commonly used)

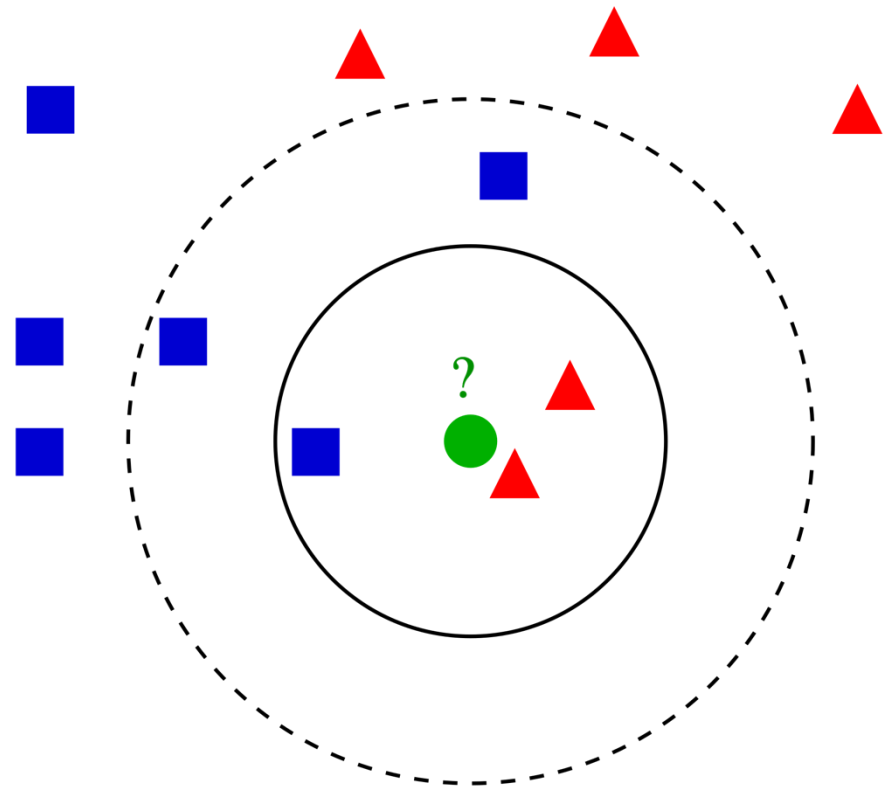
Z-SCORE



K-NEAREST NEIGHBOUR (K-NN) OUTLIER DETECTION

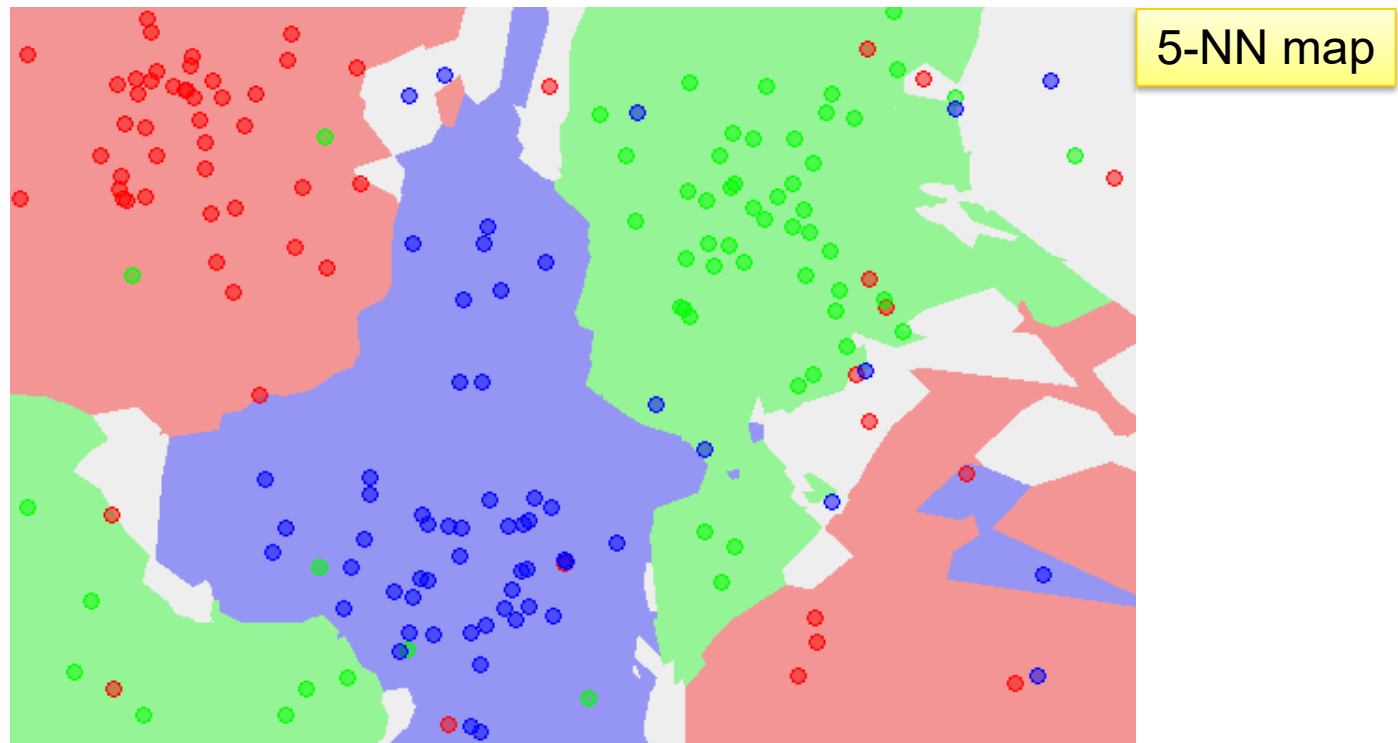
- Green: target value/object
- Classified as RED in 3-NN
- Classified as BLUE in 5-NN

Based on a distance metric and a class attribute



K-NEAREST NEIGHBOUR (K-NN) OUTLIER DETECTION

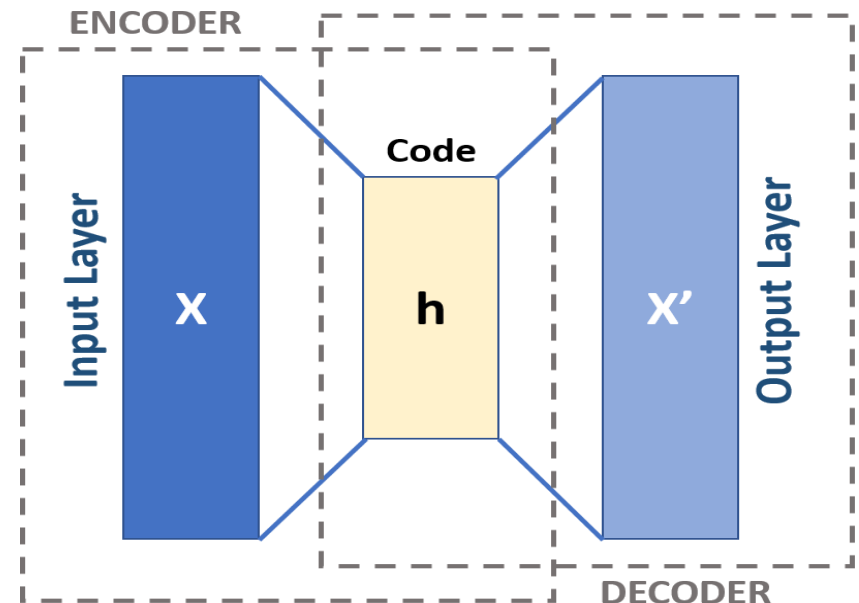
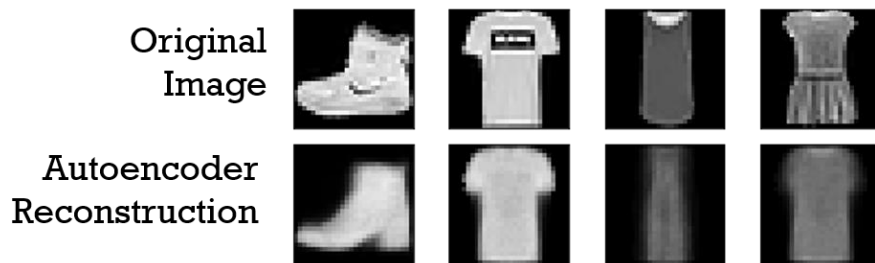
The larger the distance to the k-NN, the lower the local density, the more likely the query point is an outlier.



USING AUTOENCODERS

EFFECTIVE IN DENOISING IMAGES

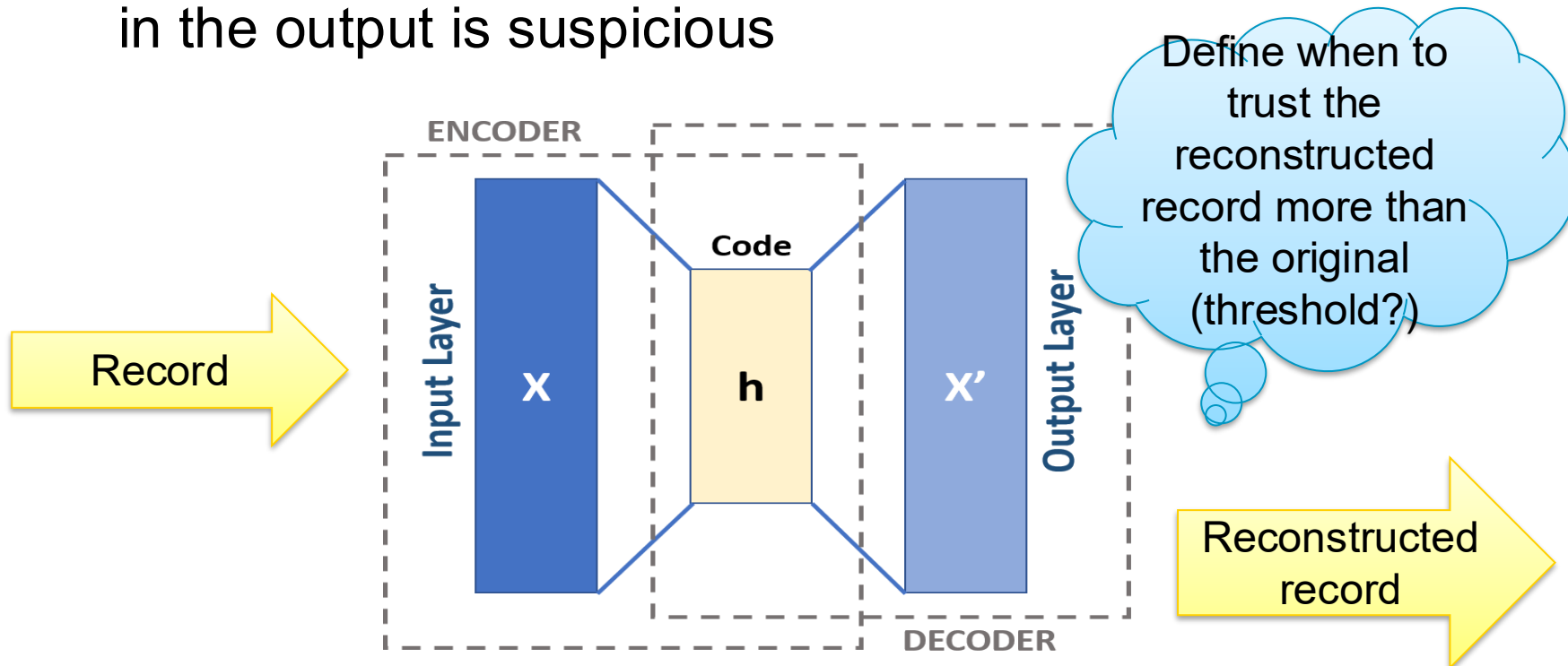
- Neural network that learns to re-construct its input
- Using a limited middle layer (code; bottleneck) it has to compress and can only represent most salient features, hence can reconstruct only normal behaviour



USING AUTOENCODERS FOR ANOMALY DETECTION

ONLY SOME RESEARCH ON THIS IDEA YET

- Train a model of a 'normal' record in the table
- Any value that is not re-constructed with a similar value in the output is suspicious

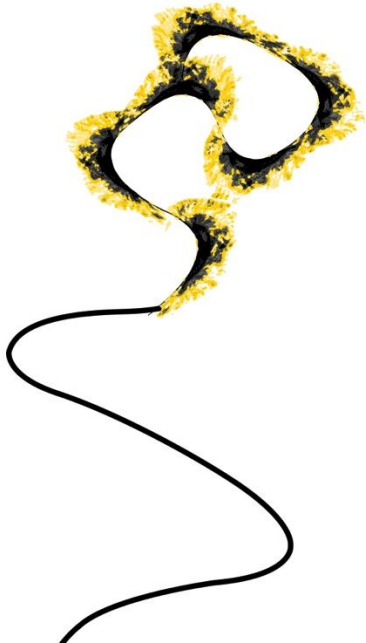


ANOMALY DETECTION / OUTLIER DETECTION

And many many
more approaches

MISSING DATA ... AND HOW TO CLEAN IT WITH DATA IMPUTATION

NOT IN THE BOOK BUT IN THE RESEARCH PAPER



SAMPLE AND POPULATION



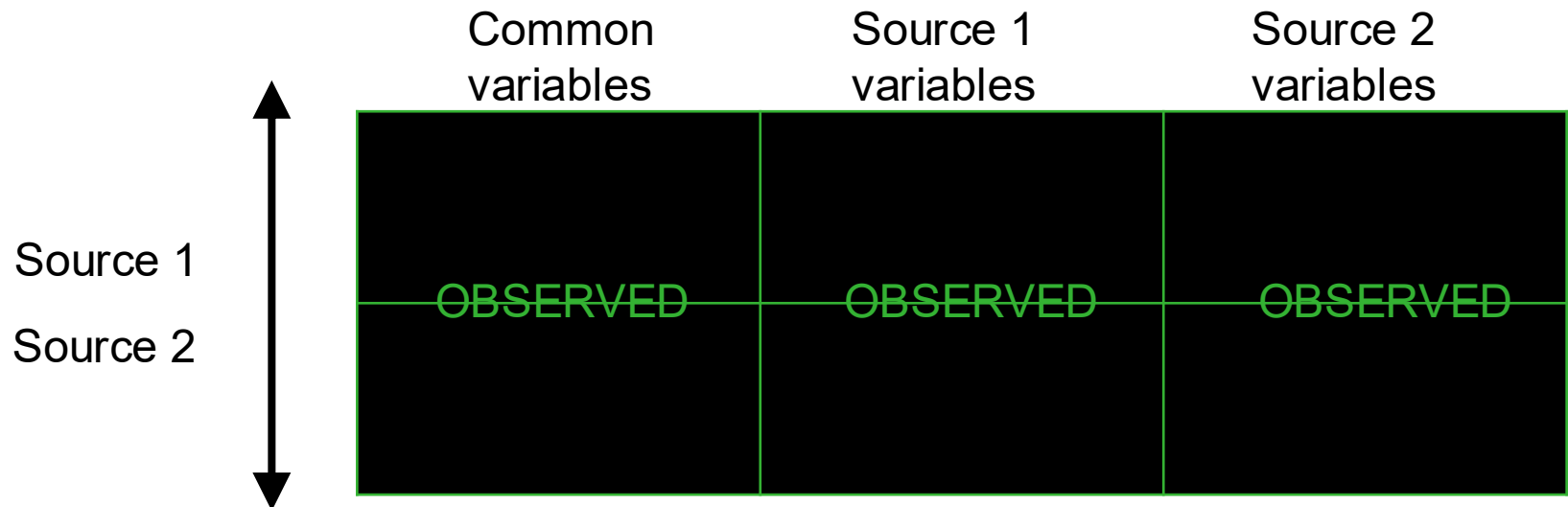
COMBINING DIFFERENT DATA SETS

TWO SOURCES WITH DATA ON THE SAME POPULATION



COMBINING DIFFERENT DATA SETS

DATA ON SAME POPULATION



Beware for entity resolution problems

GENERAL MISSING DATA PATTERN

X_1	Y_1	Y_2	Y_3	Y_4

MISSING DATA MECHANISMS

MCAR: Missing Completely at Random

- no correlation between missingness and any values, observed or missing. Random subset of the data.

MAR: Missing at Random

- systematic relationship between missing values and the observed data, but not the missing data.

MNAR: Missing Not at Random

- relationship between missings and values (typically entire rows missing not at random)

Train
model
or stat.
test on
missin
g or
not

MAR: Men are more likely to tell
you their weight than women

MNAR: Non-
respondents
to a survey

STRATEGIES FOR HANDLING MISSING DATA

- Prevention: “The best solution to handle missing data is to have none” (attributed to R.A. Fisher)
- Simple methods
 - Listwise deletion: if some attr is missing, delete row
 - Pairwise deletion: still use row for calculations where only the non-missing attrs are needed
 - Mean substitution: if missing(X) then $X := \text{mean}(X)$
 - Indicator method: add var $R := (\text{if missing}(X) \text{ then } 1 \text{ else } 0)$
 - Attribute deletion: exclude attr with too many missings
 - LOCF: Last Observation Carries Forward (time series)

ADVICE ON SIMPLE STRATEGIES

All simple methods make strong and unrealistic assumptions, for example,

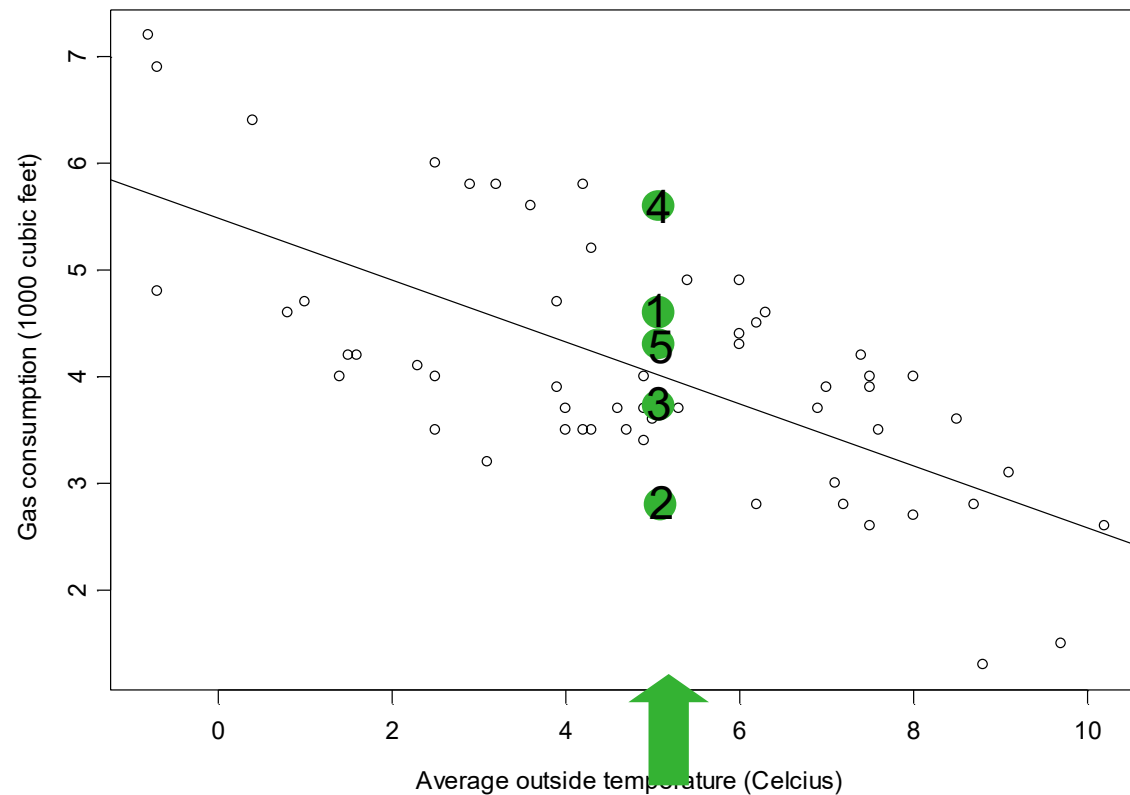
- List/pair-wise assume MCAR; otherwise bias
- Mean substitution underestimates variance and distribution is injured
- LOCF overestimates precision; biased point estimates

List-wise is least risky

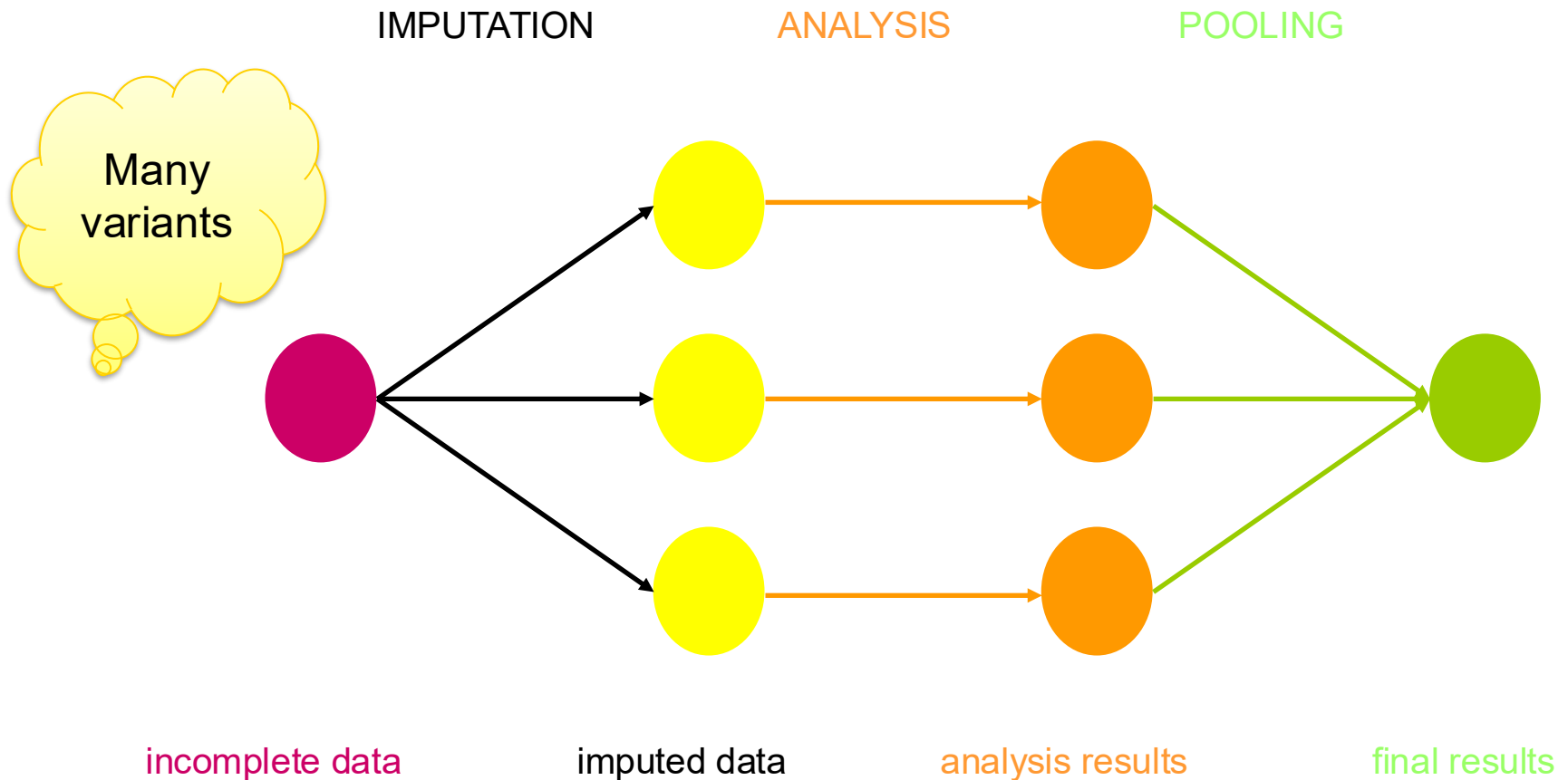
SINGLE IMPUTATION OR REGRESSION IMPUTATION

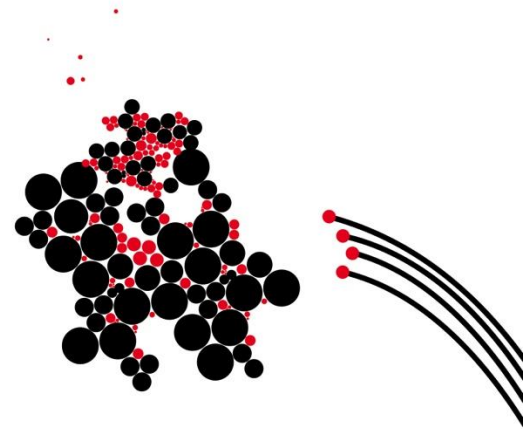
- For attribute that has missings
train regression model to predict it using the other attributes
- Then use this model's prediction for the values of the attribute that are missing

STRATEGIES FOR HANDLING MISSING DATA: MULTIPLE IMPUTATION



STRATEGIES FOR HANDLING MISSING DATA: MULTIPLE IMPUTATION





CONCLUSION

GLITCHES
CONSTRAINTS

DUPLICATES
ANOMALIES
MISSING DATA

